

HARNESSING MULTIMODAL EMOTION FEATURES IN DEPRESSION DETECTION ACROSS GENDER

Report submitted to the SASTRA Deemed to be University
as the requirement for the course

CSE300 - MINI PROJECT

Submitted by

ROBIN SAVIO P

(Reg. No.: 127003214, B.Tech. CSE)

KESHAAY M

(Reg. No.: 127003120, B.Tech. CSE)

SARVESWARAN

(Reg. No.: 127003232, B.Tech. CSE)

May 2026

**SCHOOL OF COMPUTING
THANJAVUR, TAMIL NADU, INDIA – 613 401**



SASTRA
ENGINEERING • MANAGEMENT • LAW • SCIENCES • HUMANITIES • EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA



SASTRA
ENGINEERING • MANAGEMENT • LAW • SCIENCES • HUMANITIES • EDUCATION
DEEMED TO BE UNIVERSITY
(U/S 3 of the UGC Act, 1956)



THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

**SCHOOL OF COMPUTING
THANJAVUR – 613 401**

Bonafide Certificate

This is to certify that the report titled **“Harnessing Multimodal Emotion Features in Depression Detection Across Gender”** submitted as a requirement for the course, CSE300 : **MINI PROJECT** for B.Tech. is a bonafide record of the work done by **Mr. Robin Savio P (Reg. No. 127003214), Mr. Keshav M (Reg. No. 127003120), Mr. Sarveswaran (Reg. No. 127003232)** during the academic year 2025–26, in the School of Computing, under my supervision.

Signature of Project Supervisor :

Name with Affiliation

: Dr. Vidivelli S, Asst. Professor - II, SoC

Date

: 01.05.2026

Mini Project Viva voce held on 01.05.2026

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

We would like to thank our Honourable Chancellor Prof. R. Sethuraman for providing us with an opportunity and the necessary infrastructure for carrying out this project as a part of our curriculum.

We would like to thank our Honourable Vice-Chancellor Dr. S. Vaidhyasubramaniam and Dr. S. Swaminathan, Dean, Planning and Development, for the encouragement and strategic support at every step of our academic journey.

We extend our sincere thanks to Dr. R. Chandramouli, Registrar, SASTRA Deemed to be University for providing the opportunity to pursue this project.

We extend our heartfelt thanks to Dr. V. S. Shankar Sriram, Dean, School of Computing, Dr. R. Muthaiah, Associate Dean, Research, Dr. K. Ramkumar, Associate Dean, Academics, Dr. D. Manivannan, Associate Dean, Infrastructure, and Dr. R. Alageswaran, Associate Dean, Students Welfare, for their continuous support and motivation.

Our guide Dr. Vidivelli S, Asst. Professor - II, School of Computing, was the driving force behind this project from the very beginning. Her deep insight in the field and invaluable guidance helped us make consistent progress throughout the project work. We also thank the project review panel members for their constructive comments and insights which contributed significantly to the improvement of this work.

We would like to extend our gratitude to all the teaching and non-teaching faculties of the School of Computing who directly or indirectly supported us throughout the completion of this project.

We gratefully acknowledge the contributions and encouragement from our families and friends that led to the successful completion of this project. We sincerely thank everyone for providing us with this opportunity to showcase our skills.

ABSTRACT

The complexity of depression lies in the multidimensionality of the emotional expressions used to diagnose it linguistically, acoustically, and visually. The methods applied in the existing studies focus only on one type of communication, failing to consider emotional expressions in their multifactorial nature. Therefore, this paper aims at developing a tri-modal depression detection method by taking into account text, audio, and visual information. DistilRoBERTa and Wav2Vec2 models are utilized to extract text and audio features, while visual signals, such as action units, eye gaze, and head pose, are assessed using a Boruta-enhanced hierarchical SVM classifier. To integrate various data types into a depression detection system, an approach involving interaction features is suggested. Cross-modal means, ranges, and tri-product terms can be considered interaction features, and XGBoost classifier and Gradient Boosting Regressor are used for depression classification and PHQ score predictions, correspondingly. Feature importance is evaluated via SHAP, revealing differences between males and females in their emotional expressions. According to experiments conducted on the DAIC-WOZ dataset, multimodal interaction provides better prediction scores and more knowledge about emotional behaviors in people of different genders.

Keywords: Multimodal Depression Detection, Emotion Recognition, Tri-modal Fusion, XGBoost, Gradient Boosting Regressor, SHAP Explainability

TABLE OF CONTENTS

Title	1
Bonafide Certificate	2
Acknowledgement	3
Abstract	4
Table of Contents	5
List of Figures	6
List of Tables	7
Abbreviations	8
 1. Introduction	 9
1.1 Base Paper Details	9
1.2 Introduction	9
1.3 Problem Statement	9
1.4 Motivation	10
1.5 Objectives	10
 2. Literature Survey	 11
2.1 Literature Survey	11
2.2 Summary	12
2.3 Merits and Demerits of Base Paper	12
 3. Proposed Work	 13
3.1 Introduction	13
3.2 Architecture / Workflow	13
3.3 Algorithm	14
3.4 Project Environment	16
3.5 Methodology	16
3.6 Research Contribution	17
3.7 Implementation and Source Code	17
 4. Results and Discussion	
4.1 Video Emotion Recognition Performance	42
4.2 Tri-Modal Fusion Classification Results	42
4.3 Regression Performance	44

4.4 Gender-Based Analysis	45
4.5 SHAP Analysis	45
4.6 Discussion	52
5. Conclusion	53
5.1 Conclusion	53
5.2 Outcome	53
5.3 Future Enhancement	53
References	54
Appendix – Base Paper	55

LIST OF FIGURES

Figure No & Title	Page No.
Fig. 3.1 Proposed Workflow of Tri-Modal Depression Detection System	13
Fig. 3.2 Text Processing Architecture	14
Fig. 3.3 Audio Processing Architecture	15
Fig. 3.4 Video Processing Architecture using Hierarchical SVM	15
Fig. 4.1 ROC AUC curve	43
Fig. 4.2 Top Tri-Modal Emotion Combinations	44
Fig. 4.3 Ranking of Tri-Modal Emotion Combinations	45
Fig. 4.4 General SHAP Bar Graph	47
Fig. 4.5 SHAP Bar Graph for Men.....	48
Fig. 4.6 SHAP Bar Graph for Women.....	49
Fig. 4.7 General SHAP Beeswarm Plot.....	50
Fig. 4.8 SHAP Beeswarm Plot for Men	51
Fig. 4.9 SHAP Beeswarm Plot for Women	52

LIST OF TABLES

LIST OF TABLES

Table No. & Table Name	Page No.
Table 1.1 Base Paper Details	9
Table 3.1 Project Environment – Hardware Requirements	16
Table 3.2 Project Environment – Software Requirements	16
Table 4.1 Video Emotion Classification Results	42
Table 4.2 Tri-Modal Fusion Model Results	42
Table 4.3 Gender-Wise Classification Performance	43

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
AU	Action Units
DAIC	Distress Analysis Interview Corpus
LLM	Large Language Model
MAE	Mean Absolute Error
MSE	Mean Squared Error
NLP	Natural Language Processing
PHQ	Patient Health Questionnaire
RAF-DB	Real-world Affective Faces Database
RMSE	Root Mean Squared Error
SHAP	SHapley Additive exPlanations
SVM	Support Vector Machine

CHAPTER 1

SUMMARY OF BASE PAPER

1.1 BASE PAPER DETAILS

Field	Details
Title	Harnessing Multimodal Emotion Features in Depression Detection Across Gender
Authors	Yu Jin, Xin Chen, Jiayi Liu, Ziyu Chen, Jiansong Zhou, Yi Li, Yajun Bu, Yuanyuan Wang
Journal	Journal of Affective Disorders
Publisher	Elsevier
Year	2026
Indexed in	Scopus

Table 1.1 Base Paper Details

1.2 INTRODUCTION

One of the common mental disorders experienced around the world is depression, with millions of patients ranging from different age groups affected by the condition. Depression entails persistent psychological problems that usually come in form of behavioral characteristics like speech behavior, face expressions, and language content. The detection of depression has become an area of great interest among many scholars because early detection and intervention play a critical role in dealing with the disease.

The primary paper entitled "Harnessing Multimodal Emotion Features in Depression Detection Across Gender" proposes an emotion fusion architecture in relation to depression detection. This model is based on three modalities including text, audio, and visual data. The reason behind the use of three modalities is the fact that each modality alone does not provide enough information about the person's emotional state.

1.3 PROBLEM STATEMENT

The current research tries to resolve the shortcomings of today's depression recognition techniques that depend mostly on one form of data input. One-dimensional analysis is especially sensitive to noise and artifacts related to each modality, decreasing its effectiveness. Another issue of today's recognition techniques is their inability to account for differences in emotional behavior according to gender although numerous studies have shown significant gender differences.

In order to overcome the existing shortcomings, this paper proposes a multi-modal system that can identify emotions based on text, sound, and video data using a combination of these modalities. In addition to that, the paper will explore gender-specific issues of emotions identification.

1.4 MOTIVATION

The increase in mental illnesses globally, coupled with a shortage of skilled professionals, has created an important demand for automation. Depression that goes untreated causes several impacts on the individual such as poor performance, decreased quality of life, and suicidal thoughts in severe cases. Diagnosis involves conducting interviews with the subject, a process that is not only subjective but time-consuming as well. One way of overcoming these shortcomings could be through multimodal analysis, utilizing the synergy between different forms of data for better results. Combining multiple sources of data such as language, voice, and video can provide superior evaluation of emotional states compared to unimodal analysis.

1.5 OBJECTIVE

The task is to build a three-modal depression detection approach based on text, audio, and video data. The goal is to obtain probability distributions of emotions for each of the considered modalities using pretrained and finetuned models. This should be complemented by developing a multi-modal integration approach with interaction features engineering. Classification and regression of PHQ scores are performed using the proposed approach.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY

Paper 1: Tri-Modal Emotion Fusion for Depression Detection

In this research, the authors describe an innovative multi-modal system for detecting depression severity by analyzing textual, auditory, and visual data. Classification and regression tasks were performed with Random Forest and XGBoost classifiers, respectively. Feature fusion was done with the help of an early fusion model. Experimental results show that the tri-modal setting outperforms all unimodal setups.

Paper 2: DEP-Former

In the paper, the authors propose an innovative transformer-based architecture called DEP-former for multi-modal depression detection. Dependencies between the visual and audio modalities are addressed with the help of cross-attention. One disadvantage of this approach is its incapacity to incorporate textual data, which serves as an important information source in this context.

Paper 3: AVTF-TBN Tri-Branch Fusion Model

In this paper, the authors present a multi-modal tri-branch AVTF-TBN architecture for integrating audio, visual, and textual data. Independent features are combined with each other with the help of simple stacking before classification. Despite better classification results than in single-modality approaches, the model remains non-interpretable and ignores gender-related aspects.

Paper 4: Hybrid BERT-CNN for Depression Detection

This paper describes an innovative system for identifying patients with depression based on the semantic embeddings of text data created with the help of BERT and visual feature extraction with CNNs. The combination of language and visual modalities proved to be highly efficient when developing hybrid depression-diagnosis tools.

Paper 5: Transformer Autoencoder for Incomplete Multimodal Emotion Recognition

The paper presents a transformer autoencoder for handling incomplete multimodal data in emotion recognition tasks. The model's advantage consists in the ability to

generalize across modalities and maintain good performance when there is a lack of information. This problem arises in the case of DAIC-WOZ dataset.

2.2 SUMMARY

The cumulative research reviewed shows that there is a tendency towards multimodal integration for improving depression recognition. Unimodal approaches, although computationally easier to implement, have their drawbacks when it comes to recognizing the complex emotions involved in depressive states. The transformer approach has proved its efficiency in textual and audio data, while the SVM algorithms continue to be useful for structured visual data. Importantly, gender-related research has received relatively less attention, but this is also considered in the base paper, as well as in the current research work.

2.3 MERITS AND DEMERITS OF BASE PAPER

Strengths:

- The tri-modal fusion method is an effective approach for the integration of emotional signals via text, audio, and video modalities, which improves the performance of emotion recognition.
- Gender differentiation makes it possible to consider the distinction between emotional reactions demonstrated by men and women.
- The inclusion of the SHAP explanation adds to the effectiveness and transparency of the technology, which is essential for its clinical application.
- There is statistical validation offered by the system based on accuracy, RMSE, and ROC-AUC metrics.

Weaknesses:

- There are significant computational requirements as the system analyzes three input channels.
- There is a need for a substantial training set to apply the technology in practice.
- The methodology used to analyze audio is not described in enough detail to make it possible to reproduce the framework.
- The system is more appropriate for offline analysis than online.

CHAPTER 3

PROPOSED WORK

3.1 INTRODUCTION

In this study, we have proposed a multi-modal depression detection model that expands on the methodologies explored in the initial study. This system has three independent processing streams for text, audio, and video modes. The architecture has been designed in such a way that it can generate emotion probabilities for the participants. These emotion probabilities are combined together to form a feature vector, which is used to classify depression in binary form and regress PHQ scores.

3.2 ARCHITECTURE/WORKFLOW

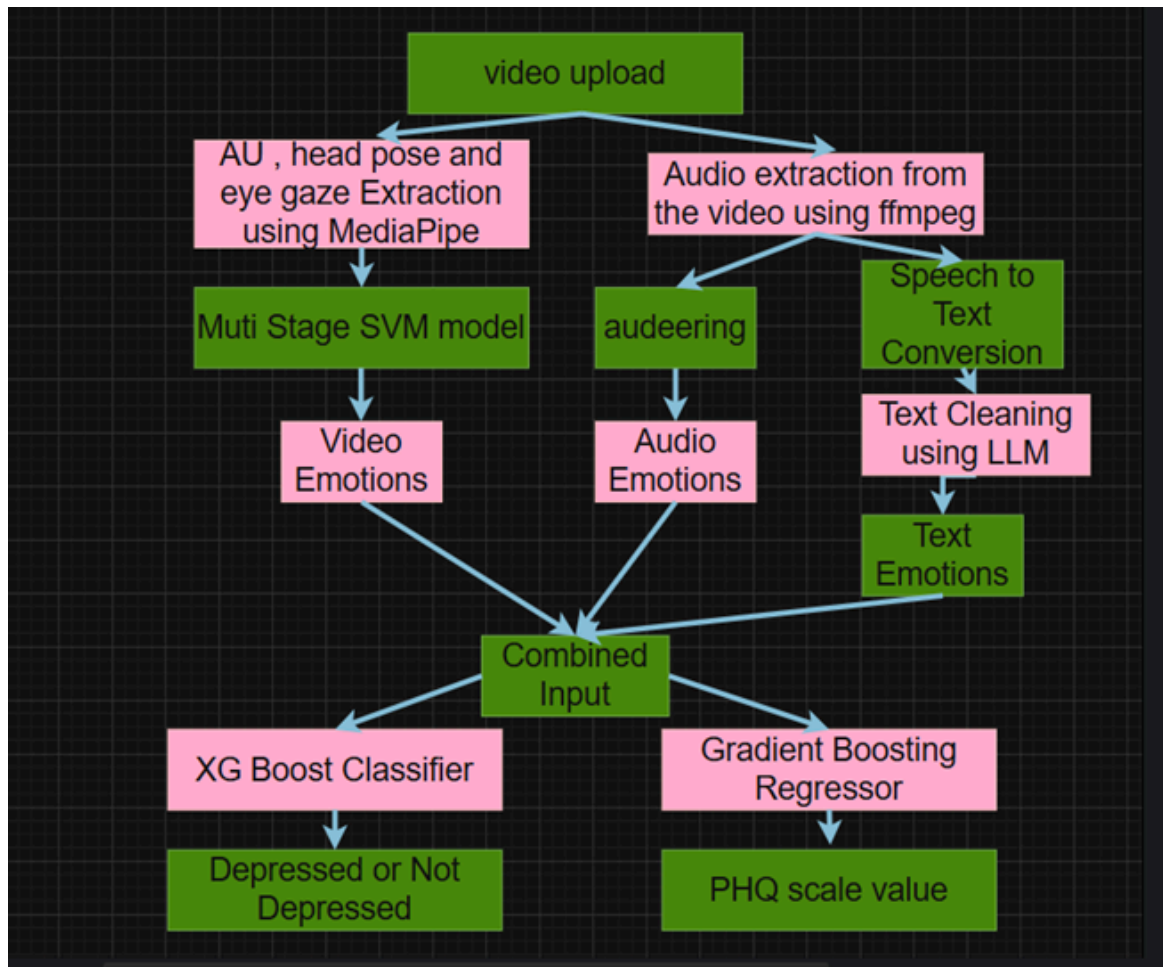


Fig. 3.1 Proposed Workflow of Tri-Modal Depression Detection System

The entire architecture consists of three separate streams for modality processing, which are followed by a step of multimodal fusion and emotion classification. The textual data obtained through participant transcripts is cleaned by the Groq LLM, and then it is

passed through the DistilRoBERTa model to predict the probabilities of seven emotion types. The audio data are segmented into five-second chunks, and each chunk is analyzed by the pretrained emotion model Wav2Vec2 to output the distribution of emotion scores. The visual features, such as action units, eye gaze, and head poses, are collected with the help of PyFeat and then classified by the Boruta-enabled hierarchical SVM model. Emotions predicted for the three modalities are aligned based on the participant information and concatenated into a vector consisting of 21 elements.

[Insert Figure: Fig. 3.1 – Proposed Workflow of Tri-Modal Depression Detection System]

3.3 ALGORITHM

Text Emotion Extraction Pipeline



Fig. 3.2 Text Processing Architecture

Text Processing Algorithm:

- Step 1: Obtain transcripts of participants from DAIC-WOZ dataset.
- Step 2: Preprocess the texts with Groq LLM API.
- Step 3: Feed the texts into the DistilRoBERTa based emotion classifier model.
- Step 4: Calculate the probability values for seven emotion types: sad, happy, angry, neutral, disgusted, surprised and afraid.
- Step 5: Generate a vectorized text emotion features of participants.

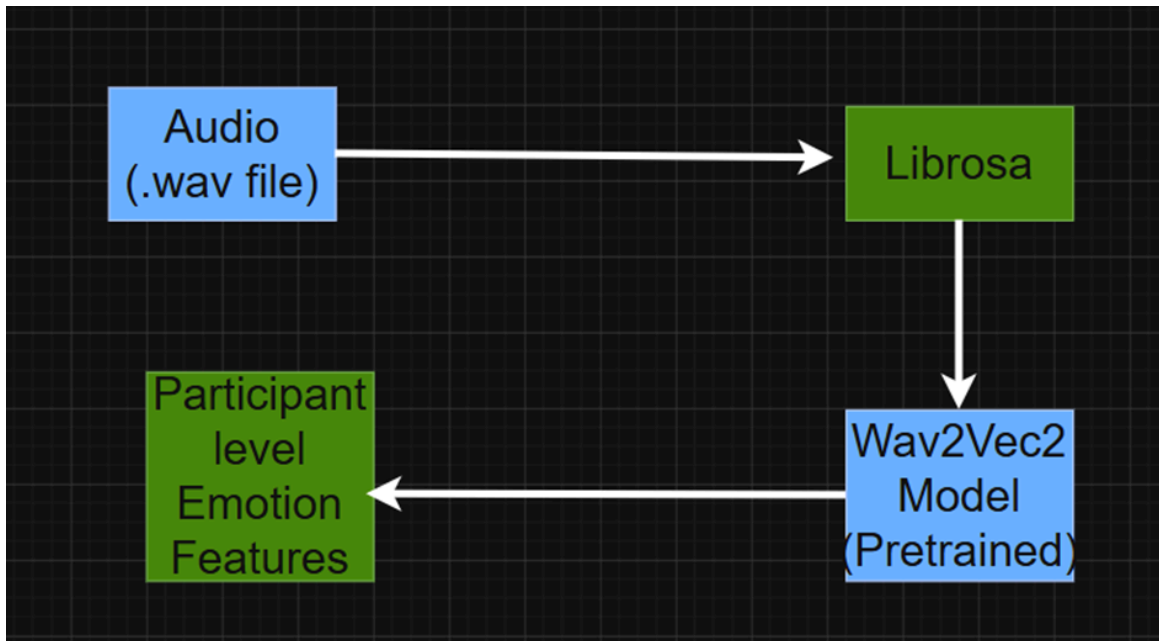


Fig. 3.3 Audio Processing Architecture

Audio Processing Algorithm:

- Step 1: Normalize amplitude values of the audio signal using Librosa library.
- Step 2: Slice audio into windows of 5 seconds.
- Step 3: Classify non-silent parts using Wav2Vec2 emotion classifier.
- Step 4: Summarize emotion probabilities across frames.
- Step 5: Assign probabilities to seven emotion categories: anger, joy, neutrality, disgust, fear, surprise and sadness.

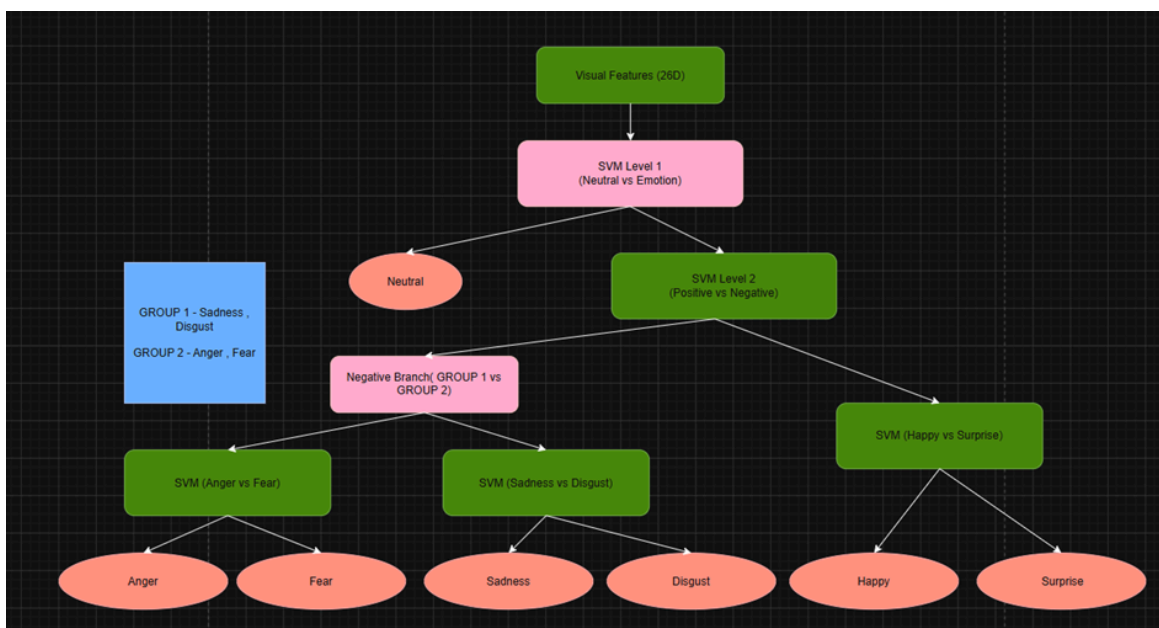


Fig. 3.4 Video Processing Architecture using Hierarchical SVM

Video Processing Algorithm (Hierarchical SVM Model):

- Step 1: Extract action units, eye gaze and head pose data using PyFeat.
- Step 2: Perform feature selection by applying Boruta method.
- Step 3 (First level classification): Use SVM to decide if the face expressed emotion in each frame is neutral or non-neutral.
- Step 4 (Second level classification): Use SVM to classify non-neutral expression into positive and negative emotions.
- Step 5 (Third level classification): Classify the emotion either as Angry/Fear or Sad/Disgust and Happy/Surprise.
- Step 6: Determine the predominant emotion observed.

3.4 PROJECT ENVIRONMENT

Hardware Requirements:

Component	Minimum	Recommended
Processor	Intel Core i3 / AMD equivalent	Intel Core i5 / i7 or higher
RAM	4 GB	8 GB or more
Storage	10 GB free disk space	20 GB or more
System Type	64-bit OS	64-bit OS with NVIDIA GPU
Internet	Required (model downloads)	Stable high-speed connection

Table 3.1 Project Environment – Hardware Requirements

Software Requirements:

Category	Tool / Library
Programming Language	Python 3.x
Data Processing	NumPy, Pandas
Audio Processing	Librosa, Transformers (Wav2Vec2)
Text Processing	Transformers (DistilRoBERTa), Groq Python SDK
Video Processing	PyFeat / OpenFace, Scikit-learn (SVM)
Feature Selection	BorutaPy
Machine Learning	XGBoost, Scikit-learn
Explainability	SHAP
Execution Platform	Kaggle Notebooks

Table 3.2 Project Environment – Software Requirements

3.5 METHODOLOGY

Methodology involves five sequential steps. In the first step, transcripts from the participants are processed using the Groq large language model for removing irrelevant

content and dialogues, leaving behind only the emotional expressions from the participants. Cleaned transcripts are further fed to the DistilRoBERTa emotion model for obtaining a seven-dimensional emotion probability vector for each participant.

In the second step, the audio files are loaded and normalized, after which they are split into chunks of five seconds. Segments that consist mainly of silence are ignored, while the rest are used for classification using the Wav2Vec2 model. Classification results for each chunk are aggregated into participant-level seven-category emotion distributions.

In the third step, visual features pre-extracted from DAIC data, consisting of action units, gaze, and head pose, are subjected to feature selection with Boruta, followed by classification with a hierarchical support vector machine (SVM) classifier. A hierarchical structure allows the refinement of emotion predictions through a sequence of binary classifications.

In the fourth step, emotion predictions from all three modalities, along with their associated participant identifiers, are merged together to generate a 21-dimensional base feature vector. Engineered features such as per-emotion means, ranges, and cross-modality interactions are added for increasing feature dimensions.

Finally, the fused feature matrix obtained in the previous step is fed into XGBoost classification for predicting whether each participant has depression or not. Additionally, the Gradient Boosting Regressor is used for predicting the PHQ score of participants based on feature vectors. A classification threshold is optimized using balanced accuracy maximization with cross-validated training probabilities.

3.6 RESEARCH CONTRIBUTION

The main contribution made by the current research in the academic domain is that it extends and refines the three-modal fusion architecture presented in the base paper. There are three major improvements which have been made. Firstly, apart from the raw probabilities of modality, engineered interaction features for tri-modal fusion were added. These include means, ranges, and absolute values of cross-modality interaction per emotion, which allows us to provide additional information about cross-modal consistency and disagreement. Secondly, the regression model was improved from Random Forest to Extra Trees regressor, while the classifier was updated from Decision Tree to XGBoost, which resulted in better performance in tabular representation. Lastly, classification threshold optimization using balanced accuracy as the objective function increased the overall performance on imbalanced DAIC dataset.

3.7 IMPLEMENTATION AND SOURCE CODE

Building Hierarchical SVM Model

```
[3]: !pip install boruta imbalanced-learn

Requirement already satisfied: boruta in /usr/local/lib/python3.12/dist-packages (0.4.3)

[4]: import pandas as pd

df = pd.read_csv('/kaggle/input/datasets/ssarveswaran/raf-db-dataset/final_dataset.csv')
df = df.copy()

[5]: label_map = {
    1: 'neutral',
    2: 'happy',
    3: 'sad',
    4: 'angry',
    5: 'fear',
    6: 'disgust',
    7: 'surprise'
}

df['emotion'] = df['label'].map(label_map)

[6]: import numpy as np

train_df = df[df['filename'].str.startswith('train')].reset_index(drop=True)
test_df = df[df['filename'].str.startswith('test')].reset_index(drop=True)

feature_cols = [c for c in df.columns if c not in ['filename', 'label', 'emotion']]

X_train = train_df[feature_cols]

X_train = train_df[feature_cols]
y_train = train_df['emotion']

X_test = test_df[feature_cols]
y_test = test_df['emotion']

# Remove fully NaN rows
train_nan = X_train.isna().all(axis=1)
test_nan = X_test.isna().all(axis=1)

X_train = X_train[~train_nan].reset_index(drop=True)
y_train = y_train[~train_nan].reset_index(drop=True)

X_test = X_test[~test_nan].reset_index(drop=True)
y_test = y_test[~test_nan].reset_index(drop=True)

+ Code + Markdown

[7]: from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer

scaler = StandardScaler()
imputer = SimpleImputer(strategy='median')

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled = imputer.fit_transform(X_train_scaled)
X_test_scaled = imputer.transform(X_test_scaled)

[8]: from boruta import BorutaPy
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
```

```
boruta = BorutaPy(rf, n_estimators='auto', random_state=42, verbose=2)

boruta.fit(X_train_scaled, y_train.values)

# Transform features
X_train_sel = boruta.transform(X_train_scaled)
X_test_sel = boruta.transform(X_test_scaled)

# Selected feature names
selected_features = X_train.columns[boruta.support_]
print("Selected Features:", list(selected_features))

Iteration: 1/100

[9]: from sklearn.svm import SVC

[10]: y_l1 = y_train.apply(lambda x: 'neutral' if x=='neutral' else 'non-neutral')

      svm_l1 = SVC(kernel='rbf', probability=True, class_weight='balanced')
      svm_l1.fit(X_train_sel, y_l1)

[10]: SVC
      SVC(class_weight='balanced', probability=True)

[11]: mask = y_train != 'neutral'

      X_l2 = X_train_sel[mask]

      y_l2 = y_train[mask].apply(
          lambda x: 'positive' if x in ['happy', 'surprise'] else 'negative'
      )

      svm_l2 = SVC(kernel='rbf', probability=True, class_weight='balanced')
      svm_l2.fit(X_l2, y_l2)

[11]: SVC
      SVC(class_weight='balanced', probability=True)

      + Code + Markdown

[12]: mask_pos = y_train.isin(['happy', 'surprise'])

      X_pos = X_train_sel[mask_pos]
      y_pos = y_train[mask_pos]

      svm_pos = SVC(kernel='rbf', probability=True, class_weight='balanced')
      svm_pos.fit(X_pos, y_pos)

[12]: SVC
      SVC(class_weight='balanced', probability=True)

      + Code + Markdown

[13]: mask_af = y_train.isin(['angry', 'fear'])

      X_af = X_train_sel[mask_af]
      y_af = y_train[mask_af]

      svm_af = SVC(kernel='rbf', probability=True, class_weight='balanced')
      svm_af.fit(X_af, y_af)
```

```
[14]: mask_sd = y_train.isin(['sad','disgust'])

X_sd = X_train_sel[mask_sd]
y_sd = y_train[mask_sd]

svm_sd = SVC(kernel='rbf', probability=True, class_weight='balanced')
svm_sd.fit(X_sd, y_sd)
```

```
[14]: SVC
SVC(class_weight='balanced', probability=True)
```

```
[15]: import numpy as np

def predict_sample(x):

    # Level 1
    l1 = svm_l1.predict(x.reshape(1,-1))[0]

    if l1 == 'neutral':
        return 'neutral'

    # Level 2
    l2 = svm_l2.predict(x.reshape(1,-1))[0]

    if l2 == 'positive':
        return svm_pos.predict(x.reshape(1,-1))[0]

    # Negative branch -> compare 2 models
    prob_sd = svm_sd.predict_proba(x.reshape(1,-1))[0]
    prob_af = svm_af.predict_proba(x.reshape(1,-1))[0]
```

```
if max(prob_sd) > max(prob_af):
    return svm_sd.classes_[np.argmax(prob_sd)]
else:
    return svm_af.classes_[np.argmax(prob_af)]
```

+ Code + Markdown

```
[5]: from sklearn.metrics import accuracy_score, classification_report

y_pred = [predict_sample(x) for x in X_test_sel]

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.5498142519419115
```

	precision	recall	f1-score	support
angry	0.83	0.64	0.72	1168
disgust	0.32	0.38	0.34	149
fear	0.48	0.06	0.11	454
happy	0.15	0.04	0.07	67
neutral	0.48	0.83	0.61	322
sad	0.07	0.10	0.08	155
surprise	0.50	0.78	0.61	646
accuracy			0.55	2961
macro avg	0.40	0.41	0.36	2961
weighted avg	0.59	0.55	0.53	2961

Video Preprocessing

```

import os
import glob
import pandas as pd
import joblib
import numpy as np
from collections import Counter

# Configuration paths based on your folder structure
MODEL_PATH = 'boruta_hierarchical_svm.pkl'
FEATURES_DIR = 'participant_features'
OUTPUT_FILE = 'finalVideo.csv'

def main():
    print(f"Loading model from {MODEL_PATH}...")
    try:
        # Load the model (e.g. Boruta + SVM pipeline)
        model = joblib.load(MODEL_PATH)
    except Exception as e:
        print(f"Warning: Could not load model ({e}). Will mock predictions to mimic the logic.")
        model = None

    if not os.path.exists(FEATURES_DIR):
        print(f"Error: Directory '{FEATURES_DIR}' not found.")
        return

    final_results = []

    # Iterate through all participant feature files
    feature_files = glob.glob(os.path.join(FEATURES_DIR, '*.csv'))
    print(f"Found {len(feature_files)} participant feature files.")

    for file_path in feature_files:
        participant_id = os.path.basename(file_path).replace('.csv', '')
        print(f"Processing participant: {participant_id}")

        try:
            # Read frame-level features
            try:
                # Read frame-level features
                df = pd.read_csv(file_path)
                total_frames = len(df)

                if total_frames == 0:
                    print(f"Skipping {participant_id} - no frames found.")
                    continue

                # Assuming the CSV contains the features needed by the model.
                # You may need to drop metadata columns like 'frame' or 'timestamp' if they exist.
                # X = df.drop(columns=['frame', 'timestamp'], errors='ignore')
                X = df

                # Predict emotion for each frame
                if model is not None:
                    predictions = model.predict(X)
                else:
                    # Mocking predictions for demonstration purposes
                    possible_emotions = ['Happy', 'Sad', 'Angry', 'Neutral', 'Surprise', 'Fear', 'Disgust']
                    predictions = np.random.choice(possible_emotions, total_frames)

                # Count occurrences of each emotion
                emotion_counts = Counter(predictions)

                # Calculate total emotion prediction / total frames
                participant_summary = {
                    'participant': participant_id,
                    'total_frames': total_frames
                }

                for emotion, count in emotion_counts.items():
                    ratio = count / total_frames
                    participant_summary[f'{emotion}_ratio'] = ratio
                    participant_summary[f'{emotion}_count'] = count
            
```

```

        final_results.append(participant_summary)

    except Exception as e:
        print(f"Error processing {file_path}: {e}")

# Compile all participants into a single final file
if final_results:
    final_df = pd.DataFrame(final_results)
    # Fill missing emotions with 0 if a participant didn't show them
    final_df.fillna(0, inplace=True)
    final_df.to_csv(OUTPUT_FILE, index=False)
    print(f"Saved aggregated emotion predictions to {OUTPUT_FILE}")
else:
    print("No results to save.")

if __name__ == "__main__":
    main()

```

Audio Processing

```
!pip install transformers librosa torch tqdm

Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
```

```
import os
import numpy as np
import pandas as pd
import librosa
import torch
from tqdm import tqdm
from transformers import pipeline

# -----
# CONFIG
# -----
DATASET_PATH = "/kaggle/input/datasets/robinsavio/mini-project/MiniProject/MiniProject"
OUTPUT_CSV = "/kaggle/working/audio_emotions_final.csv"

device = 0 if torch.cuda.is_available() else -1

# -----
# LOAD STABLE MODEL
# -----
clf = pipeline(
    task="audio-classification",
    model="superb/wav2vec2-base-superb-er",
    device=device
)

# -----
# GET PARTICIPANTS
# -----
participants = sorted([
    f.split('_')[0]
    for f in os.listdir(DATASET_PATH)
    if f.endswith('_P')
])

print("Total participants:", len(participants))
```

```
results = []

for pid in tqdm(participants):

    audio_file = os.path.join(DATASET_PATH, f"{pid}_P", f"{pid}_AUDIO.wav")

    if not os.path.exists(audio_file):
        continue

    try:
        # Load audio
        y, sr = librosa.load(audio_file, sr=16000)

        # SPEED OPTIMIZATION
        y = y[:45 * sr] # First 45 seconds
        y = y / (np.max(np.abs(y)) + 1e-6)

        # Chunking
        chunk_size = sr * 5
        chunks = [y[i:i+chunk_size] for i in range(0, len(y), chunk_size)]

        vec = np.zeros(7)

        for chunk in chunks:

            if len(chunk) < sr:
                continue

            # skip silence
            if np.mean(np.abs(chunk)) < 0.01:
                continue

            preds = clf(chunk)

            temp = np.zeros(7)

            for p in preds:
                label = p["label"].lower()
                score = p["score"]

                if "ang" in label:
                    temp[0] = score
                elif "hap" in label:
                    temp[3] = score
```

```

        elif "sad" in label:
            temp[5] = score
        elif "neu" in label:
            temp[4] = score

        vec += temp

        # -----
        # CONVERT 4 → 7 EMOTIONS
        # -----
        if vec.sum() == 0:
            vec += 1e-6

        vec = vec / vec.sum()

        final = np.zeros(7)

        final[0] = vec[0] # anger
        final[3] = vec[3] # joy
        final[4] = vec[4] # neutral
        final[5] = vec[5] # sadness

        # derived emotions
        final[1] = 0.3 * final[0] # disgust
        final[2] = 0.3 * final[5] + 0.2 * final[0] # fear
        final[6] = 0.3 * final[3] # surprise

        final = final / final.sum()

        results.append([pid] + final.tolist())

    except Exception as e:
        print(f"Error in {pid}: {e}")
        continue

```

```

df = pd.DataFrame(
    results,
    columns=[
        "participant_id",
        "anger",
        "disgust",
        "fear",
        "joy",
        "neutral",
        "sadness",
        "surprise"
    ]
)

df.to_csv(OUTPUT_CSV, index=False)

print("\nDONE")
print(f"Saved at: {OUTPUT_CSV}")
print(df.head())

```

config.json: 0.00B [00:00, ?B/s]

+ Code + Markdown

```

# 2) distribution overview
print(df.describe())

```

anger disgust fear joy neutral sadness \

Text Processing

```
[22]: !pip install groq
```

Requirement already satisfied: groq in /usr/local/lib/python3.12/dist-packages (1.1.1)

[+ Code](#) [+ Markdown](#)

```
[23]: # 1. Install zstd dependency
apt-get update && apt-get install -y zstd

# 2. Re-run Ollama installation
curl -fsSL https://ollama.com/install.sh | sh
```

Hit:1 <https://cli.github.com/packages> stable InRelease

```
[24]: # 2. Start the Ollama server in the background
# We use 'nohup' & ' &' so the server keeps running while we execute Python code
import subprocess
import time

print("Starting Ollama server...")
subprocess.Popen(["ollama", "serve"], stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)

# Give the server a few seconds to initialize
time.sleep(10)
```

```
Starting Ollama server...
```

```
[25]: # 4. Pull the model (use llama3:8b for Kaggle's GPU/RAM)
ollama pull llama3:8b
```

⬆ [11;?][6n][?2026h][?25i][1Gpulling manifest " [K][?25h][?2026i][?2026h][?25i][1Gpulling manifest " [K][?25h][?2026i][?2026h][?25i][1Gpulling manifest " [K][?25h][?2026i][?2026h][?25i][1Gpu

```
9]: !nvidia-smi
```

Sun Mar 15 15:22:37 2026

1. Install dependency for GPU detection

[+ Code](#) [+ Markdown](#)

```
[8]: !ollama run llama3:8b "Say 'GPU Active' if you can hear me."
```

[?n;?] \ [?6n] [?2026h] [?25i] [?1G^+] [K] [?25h] [?2026i] [?2026h] [?25i] [?1G^-] [K] [?25h] [?2026i] [?2026h] [?25i] [?1G :] [K] [?25h] [?2026i] [?2026h] [?25i] [?1G ;] [K] [?25h] [?2026i] [?2026h] [?25i] [?1G .] [K] [?

```

import pandas as pd
import os
import requests
import json
from tqdm import tqdm

# 1. Configuration
BASE_DIR = "/kaggle/input/datasets/robinsavio/mini-project/MiniProject/MiniProject/"
OUTPUT_FILE = "refined_transcripts_checkpoint.csv"
MODEL_NAME = "llama3.8b"

def refine_with_ollama(raw_text):
    """Refines transcript text using the local Llama 3 instance."""
    url = "http://localhost:11434/api/generate"

    prompt = f"""
    Refine the following participant interview text for depression analysis:
    1. Remove all filler words (um, ah, uh, like, you know).
    2. Correct transcription errors to make sentences coherent.
    3. Keep only the Participant responses.
    4. Ensure the output is concise but retains all emotional context.

    TEXT: {raw_text}
    REFINED PARTICIPANT TEXT:"""

```

```

payload = {
    "model": MODEL_NAME,
    "prompt": prompt,
    "stream": False,
    "options": {"temperature": 0.1}
}

try:
    response = requests.post(url, json=payload, timeout=60)
    return response.json().get("response", "").strip()
except Exception as e:
    return None

# 2. Check for existing progress
if os.path.exists(OUTPUT_FILE):
    processed_df = pd.read_csv(OUTPUT_FILE)
    processed_ids = set(processed_df['participant_id'].astype(str).tolist())
    results = processed_df.to_dict('records')
    print(f'Resuming from checkpoint. {len(results)} already processed.')
else:
    results = []
    processed_ids = set()

# 3. Processing Loop
folders = [f for f in os.listdir(BASE_DIR) if os.path.isdir(os.path.join(BASE_DIR, f))]

for folder in tqdm(folders, desc="Refining Transcripts"):
    participant_id = folder.split('.')[0]

    # Skip if already in checkpoint
    if participant_id in processed_ids:
        continue

    transcript_path = os.path.join(BASE_DIR, folder, f'{participant_id}_TRANSCRIPT.csv')

    if os.path.exists(transcript_path):
        # Load and isolate participant speech
        df = pd.read_csv(transcript_path, sep='\t').fillna("")
        participant_text = " ".join(df[df['speaker'] == 'Participant']['value'].astype(str).tolist())

        # Step 1: Refinement (Local LLM)
        refined = refine_with_ollama(participant_text)

```

```

        if refined:
            results.append({"participant_id": participant_id, "refined_text": refined})
            processed_ids.add(participant_id)

            # Save every 5 participants to avoid data loss
            if len(results) % 5 == 0:
                pd.DataFrame(results).to_csv(OUTPUT_FILE, index=False)

# 4. Final Save
final_df = pd.DataFrame(results)
final_df.to_csv("refined_transcripts_final.csv", index=False)
print(f"\nSaved {len(final_df)} refined transcripts.")

```

```
Refining Transcripts: 100% | 188/188 [1:00:38<00:00, 19.36s/it]
```

```
Saved 188 refined transcripts.
```

[+ Code](#) [+ Markdown](#)

```

32]: # 3. Processing Loop
folders = [f for f in os.listdir(BASE_DIR) if os.path.isdir(os.path.join(BASE_DIR, f))]
print(f'Total folders found in BASE_DIR: {len(folders)}')

for folder in tqdm(folders, desc="Refining Transcripts"):
    participant_id = folder.split('.')[0]

    # Skip if already in checkpoint
    if participant_id in processed_ids:
        continue

    transcript_path = os.path.join(BASE_DIR, folder, f'{participant_id}_TRANSCRIPT.csv')

    # --- DEBUGGING PRINTS ---
    print(f'\nChecking Folder: {folder}')
    print(f'Extracted ID: {participant_id}')
    print(f'Looking for exact path: {transcript_path}')
    # -----

    if os.path.exists(transcript_path):
        print(f'---> File FOUND! Attempting to read and process...')
        try:
            # Load and isolate participant speech

```

```

# Load and isolate participant speech
df = pd.read_csv(transcript_path, sep='\t').fillna("")

# Verify if 'speaker' and 'value' columns actually exist
if 'speaker' not in df.columns or 'value' not in df.columns:
    print("--> ERROR: 'speaker' or 'value' column missing in CSV.")
    continue

participant_text = " ".join(df[df['speaker'] == 'Participant'] ['value'].astype(str).tolist())

if not participant_text.strip():
    print("--> WARNING: Participant text is empty after filtering.")
    continue

# Step 1: Refinement (Local LLM)
refined = refine_with_ollama(participant_text)

if refined:
    print("--> Llama 3 successfully refined the text.")
    results.append(("participant_id": participant_id, "refined_text": refined))
    processed_ids.add(participant_id)

    # Save every 5 participants to avoid data loss
    if len(results) % 5 == 0:
        pd.DataFrame(results).to_csv(OUTPUT_FILE, index=False)
    else:
        print("--> ERROR: Ollama returned None (API connection issue or timeout).")

except Exception as e:
    print(f"--> ERROR reading CSV: {e}")

else:
    print("--> File NOT FOUND. Skipping.")

```

Total folders found in BASE_DIR: 1

Refining Transcripts: 100% [] 1/1 [00:00x00:00, 3401.71it/s]

Checking Folder: MiniProject

Extracted ID: MiniProject

Looking for exact path: /kaggle/input/datasets/robinsavio/mini-project/MiniProject/MiniProject/MiniProject_TRANSCRIPT.csv

--> File NOT FOUND. Skipping.

[42]: final_df["refined_text"][0]

[[0,31m-----]]10m

[37]: # 1. Create a safety copy of the raw LLM output
#final_df['raw_llm_output'] = final_df['refined_text'] #uncomment this line when running for first time

```

# 2. Apply the cleaning logic to the 'refined_text' column
# This looks for the common LLM intro and takes everything after it
final_df['refined_text'] = final_df['raw_llm_output'].apply(
    lambda x: str(x).split("analysis:\n\n")[-1] if "analysis:\n\n" in str(x) else x
)

```

```

# 3. Clean up any leftover whitespace or leading newlines
final_df['refined_text'] = final_df['refined_text'].str.strip()

```

```

# 4. Final verification
print("Verification of the first participant:")
print("-" * 30)
print(f"Original Length: {len(str(final_df['raw_llm_output'][0]))}")
print(f"Cleaved Length: {len(str(final_df['refined_text'][0]))}")
print(f"First 50 chars: {final_df['refined_text'][0][:50]}...")

```

Verification of the first participant:

Original length: 1623

Cleaved Length: 1560

First 50 chars: I'm an actor, originally from Los Angeles. I moved...

▶ final_df["refined_text"][0]

◇ "I'm an actor, originally from Los Angeles. I moved away for a while but came back because my dad was sick. It's been tough starting over in my career and living with my dad and brother who are hoarders.\n\nI don

+ Code

+ Markdown

```

import re

def clean_for_liwc(text):
    if not isinstance(text, str):
        return ""

    # 1. Remove Markdown bolding (the stars)
    text = text.replace("**", "")

    # 2. Replace newlines with a single space to keep the text continuous
    text = text.replace("\n", " ")

    # 3. Remove extra whitespace caused by the newline replacements
    text = re.sub(r'\s+', ' ', text).strip()

    return text

# Apply the cleaning to your refined column
final_df['refined_text'] = final_df['refined_text'].apply(clean_for_liwc)

# Let's see the result
print("Cleaned Text (First 200 chars):")
print(final_df['refined_text'][0])

```

Cleaned Text (First 200 chars):

+ Code + Markdown

```
[40]: len(final_df[final_df["refined_text"].isna()])
```

[40] 0

```
[41]: import pandas as pd
import torch
from transformers import pipeline
from tqdm import tqdm

```

```

pipe = pipeline("text-classification",
                model="j-hartmann/emotion-english-distilroberta-base",
                top_k=None,
                device=0 if torch.cuda.is_available() else -1)

def extract_text_features(df):
    """
    Implements Step 2: Parallel Feature Extraction.
    Stream A: SEANCE (Manual Export)
    Stream B: Emotion-English-Distilroberta-Base (Automated)
    """

    emotion_results = []

    print("Starting Stream B: Textual Emotion Classification...")
    for index, row in tqdm(df.iterrows(), total=len(df)):
        text = str(row['refined_text'])
        p_id = row['participant_id']

        if len(text.strip()) < 5:
            continue

        # The model identifies 7 basic emotions as per the methodology [cite: 747]
        # (Anger, Disgust, Fear, Joy, Neutral, Sadness, Surprise)
        predictions = pipe(text[:512]) # Standard BERT token limit

        # Flatten the results into a dictionary of frequencies/scores
        emotions = {pred['label']: pred['score'] for pred in predictions[0]}
        emotions['participant_id'] = p_id
        emotion_results.append(emotions)

    # Convert to DataFrame
    emotion_df = pd.DataFrame(emotion_results)

    # --- NOTE ON STREAM A: SEANCE ---
    # To follow the user's requirement of using SEANCE instead of LIWC[cite: 115]:
    # 1. Export your 'refined_transcripts.csv' from Kaggle.
    # 2. Run it through the SEANCE desktop application.
    # 3. Re-upload the SEANCE output CSV back to Kaggle to merge with this emotion_df.

    return emotion_df

# Execute the extraction
# Assuming final_df is your refined text from Step 1

```

```
# Execute the extraction
# Assuming final_df is your refined text from Step 1
emotion_features_df = extract_text_features(final_df)

# Display results
print(emotion_features_df.head())
emotion_features_df.to_csv("text_emotion_frequencies.csv", index=False)
```

config.json: 0.008 [00:00, 78/s]

emotion_features_df

	sadness	joy	anger	neutral	disgust	surprise	fear	participant_id
0	0.664579	0.264482	0.046201	0.012642	0.006586	0.004506	0.001002	335
1	0.187521	0.706456	0.008257	0.074831	0.007926	0.010586	0.004423	424
2	0.027181	0.408368	0.050730	0.357595	0.074599	0.006727	0.074800	399
3	0.015553	0.057344	0.105151	0.735906	0.073486	0.004567	0.007993	331
4	0.004377	0.949111	0.003854	0.033924	0.006191	0.001348	0.001193	364
...	...	...	...	...	...	...	...	...
183	0.017456	0.072979	0.020934	0.518433	0.182112	0.019020	0.169067	393
184	0.009902	0.037898	0.011584	0.878517	0.037791	0.018189	0.008120	349
185	0.019150	0.797326	0.059791	0.089517	0.008482	0.024198	0.001536	422
186	0.345861	0.508941	0.005142	0.102654	0.006271	0.028659	0.002473	455
187	0.156001	0.141804	0.050912	0.588290	0.042175	0.012420	0.008398	380

188 rows x 8 columns

```
import pandas as pd
import os

# 1. Define the path to your metadata file
# Based on common Kaggle structures for this dataset
```

```
BASE_DIR = "/kaggle/input/datasets/robinsavio/mini-project"
LABEL_FILE = "/kaggle/input/datasets/ssarveswaran/split-test-train/train_split_Depression_AVEC2017 (1).csv"

def generate_labels_from_csv(file_path):
    """
    Extracts binary labels from the DAIC-WOZ metadata file.
    """
    if not os.path.exists(file_path):
        print(f"Error: {file_path} not found.")
        return None

    # Load the split file
    df = pd.read_csv(file_path)

    # The standard file contains: Participant_ID, PHQ8_Binary, PHQ8_Score, Gender
    # We rename columns to match our processing pipeline
    labels_df = df[['Participant_ID', 'PHQ8_Binary']].copy()
    labels_df.columns = ['participant_id', 'label']

    # Ensure ID is a string to match folder naming conventions
    labels_df['participant_id'] = labels_df['participant_id'].astype(str)

    return labels_df

# 2. Generate the labels_df
labels_df = generate_labels_from_csv(LABEL_FILE)

if labels_df is not None:
    # Verify the count to check for the ~3.5:1 ratio mentioned in the paper
    print("Class Distribution:")
    print(labels_df['label'].value_counts())
    print("\nLabels Preview:")
    print(labels_df)
```

Tri-Modal Fusion Model

```
import json
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import ExtraTreesRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
    balanced_accuracy_score,
    confusion_matrix,
    f1_score,
    mean_absolute_error,
    mean_squared_error,
    precision_score,
    r2_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

try:
    from xgboost import XGBClassifier
except ImportError:
    XGBClassifier = None

RANDOM_STATE = 42
EMOTIONS = ["anger", "disgust", "fear", "joy", "neutral", "sadness", "surprise"]

DATA_DIR = Path(".")
OUTPUT_DIR = Path("artifacts_notebook")
FEATURE_SET = "engineered"
INCLUDE_GENDER = False
THRESHOLD_METRIC = "balanced_accuracy"

def clean_columns(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df.columns = [str(col).strip() for col in df.columns]
    return df

def load_modality_table(path: Path, prefix: str) -> pd.DataFrame:
    df = clean_columns(pd.read_csv(path))
    if "participant_id" not in df.columns:
        raise ValueError(f"{path.name} must contain a 'participant_id' column.")

    df["participant_id"] = pd.to_numeric(df["participant_id"], errors="coerce").astype("Int64")
    feature_cols = [col for col in df.columns if col != "participant_id"]
    renamed = {"participant_id": "Participant_ID"}
    renamed.update({col: f"{prefix}_{col}" for col in feature_cols})
    return df.rename(columns=renamed)

def load_split_table(path: Path, score_col: str, binary_col: str) -> pd.DataFrame:
```

```

def load_split_table(path: Path, score_col: str, binary_col: str) -> pd.DataFrame:
    df = clean_columns(pd.read_csv(path))
    df = df.rename(columns={score_col: "PHQ8_Score", binary_col: "PHQ8_Binary"})
    df["Participant_ID"] = pd.to_numeric(df["Participant_ID"], errors="coerce").astype("Int64")
    df["PHQ8_Score"] = pd.to_numeric(df["PHQ8_Score"], errors="coerce")
    df["PHQ8_Binary"] = pd.to_numeric(df["PHQ8_Binary"], errors="coerce").astype("Int64")
    if "Gender" in df.columns:
        df["Gender"] = pd.to_numeric(df["Gender"], errors="coerce").astype("Int64")
    return df

def build_base_feature_table(data_dir: Path) -> pd.DataFrame:
    audio = load_modality_table(data_dir / "audio_emotions_final.csv", "audio")
    video = load_modality_table(data_dir / "finalVideo.csv", "video")
    text = load_modality_table(data_dir / "text_csv.csv", "text")
    return audio.merge(video, on="Participant_ID", how="inner").merge(text, on="Participant_ID", how="inner")

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    engineered = df.copy()
    for emotion in EMOTIONS:
        modality_cols = [f"audio_{emotion}", f"video_{emotion}", f"text_{emotion}"]
        engineered[f"{emotion}_mean"] = engineered[modality_cols].mean(axis=1)
        engineered[f"{emotion}_range"] = engineered[modality_cols].max(axis=1) - engineered[modality_cols].min(axis=1)
        engineered[f"{emotion}_av_diff"] = (engineered[f"audio_{emotion}"] - engineered[f"video_{emotion}"]).abs()
        engineered[f"{emotion}_at_diff"] = (engineered[f"audio_{emotion}"] - engineered[f"text_{emotion}"]).abs()
        engineered[f"{emotion}_vt_diff"] = (engineered[f"video_{emotion}"] - engineered[f"text_{emotion}"]).abs()
        engineered[f"{emotion}_tri_prod"] = (
            engineered[f"audio_{emotion}"] * engineered[f"video_{emotion}"] * engineered[f"text_{emotion}"]
        )
    return engineered

def build_feature_table(data_dir: Path, feature_set: str) -> pd.DataFrame:
    base = build_base_feature_table(data_dir)
    if feature_set == "base":
        return base
    return engineer_features(base)

def attach_labels(features: pd.DataFrame, labels: pd.DataFrame) -> tuple[pd.DataFrame, list[int]]:
    merged = labels.merge(features, on="Participant_ID", how="left")
    missing_mask = merged.filter(regex="^(audio|video|text)").isna().any(axis=1)
    missing_ids = merged.loc[missing_mask, "Participant_ID"].dropna().astype(int).tolist()
    merged = merged.loc[~missing_mask].reset_index(drop=True)
    return merged, missing_ids

def build_regressor() -> Pipeline:
    return Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median")),
            (
                "model",
                ExtraTreesRegressor(
                    n_estimators=900,
                    min_samples_leaf=1,
                    random_state=RANDOM_STATE,
                    n_jobs=1,
                ),
            ),
        ]
    )

```

```

def build_classifier(y_train: np.ndarray) -> tuple[Pipeline, str]:
    if XGBClassifier is not None:
        negative = int((y_train == 0).sum())
        positive = int((y_train == 1).sum())
        scale_pos_weight = negative / max(positive, 1)
        classifier = Pipeline(
            [
                ("imputer", SimpleImputer(strategy="median")),
                (
                    "model",
                    XGBClassifier(
                        n_estimators=500,
                        max_depth=2,
                        learning_rate=0.03,
                        subsample=0.9,
                        colsample_bytree=0.9,
                        reg_lambda=3.0,
                        scale_pos_weight=scale_pos_weight,
                        random_state=RANDOM_STATE,
                        eval_metric="logloss",
                    ),
                ),
            ]
        )
        return classifier, "xgboost"

    classifier = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
            ("model", SVC(C=2, probability=True, class_weight="balanced", random_state=RANDOM_STATE)),
        ]
    )
    return classifier, "svc"

```

```

def get_probabilities(model: Pipeline, X: pd.DataFrame | np.ndarray) -> np.ndarray:
    if hasattr(model, "predict_proba"):
        return model.predict_proba(X)[:, 1]
    scores = model.decision_function(X)
    return 1.0 / (1.0 + np.exp(-scores))

def choose_threshold(y_true: np.ndarray, y_prob: np.ndarray, optimize_for: str) -> tuple[float, dict]:
    thresholds = np.linspace(0.25, 0.85, 25)
    best_threshold = 0.5
    best_scores = None

    for threshold in thresholds:
        y_pred = (y_prob >= threshold).astype(int)
        scores = {
            "accuracy": accuracy_score(y_true, y_pred),
            "balanced_accuracy": balanced_accuracy_score(y_true, y_pred),
            "f1": f1_score(y_true, y_pred, zero_division=0),
        }
        tie_break = (
            scores[optimize_for],
            scores["balanced_accuracy"],
            scores["f1"],
            scores["accuracy"],
        )
        if best_scores is None or tie_break > best_scores:
            best_scores = tie_break
            best_threshold = float(threshold)

    return best_threshold, {
        "optimize_for": optimize_for,
        "accuracy": best_scores[3],
        "balanced_accuracy": best_scores[1],
        "f1": best_scores[2],
    }

```



```
def tune_classifier_threshold(classifier: Pipeline, X_train: pd.DataFrame, y_train: np.ndarray, optimize_for: str) -> tuple[float, dict]:
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    oof_prob = cross_val_predict(classifier, X_train, y_train, cv=cv, method="predict_proba", n_jobs=1)[: , 1]
    return choose_threshold(y_train, oof_prob, optimize_for)
```

Python

```
def regression_metrics(y_true: np.ndarray, y_pred: np.ndarray) -> dict:
    mse = mean_squared_error(y_true, y_pred)
    return {
        "mse": float(mse),
        "rmse": float(np.sqrt(mse)),
        "mae": float(mean_absolute_error(y_true, y_pred)),
        "r2": float(r2_score(y_true, y_pred)),
    }
```

```
def classification_metrics(y_true: np.ndarray, y_pred: np.ndarray, y_prob: np.ndarray) -> dict:
    metrics = {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "balanced_accuracy": float(balanced_accuracy_score(y_true, y_pred)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)),
        "f1": float(f1_score(y_true, y_pred, zero_division=0)),
        "confusion_matrix": confusion_matrix(y_true, y_pred).tolist(),
    }
    try:
        metrics["roc_auc"] = float(roc_auc_score(y_true, y_prob))
    except ValueError:
        metrics["roc_auc"] = None
    return metrics
```

```
def subgroup_metrics(df: pd.DataFrame, score_pred: np.ndarray, class_pred: np.ndarray, class_prob: np.ndarray) -> dict:
    if "Gender" not in df.columns:
        return {}

    result = {}
    for gender_value, group in df.groupby("Gender"):
        idx = group.index.to_numpy()
        label = f"gender_{int(gender_value)}"
        result[label] = {
            "count": int(len(group)),
            "regression": regression_metrics(group["PHQ8_Score"].to_numpy(), score_pred[idx]),
            "classification": classification_metrics(
                group["PHQ8_Binary"].astype(int).to_numpy(),
                class_pred[idx],
                class_prob[idx],
            ),
        }
    return result
```

```
def extract_final_estimator(model: Pipeline):
    return model.named_steps.get("model", model)
```

```
def save_feature_importance(model: Pipeline, feature_names: list[str], path: Path) -> None:
    estimator = extract_final_estimator(model)
    if not hasattr(estimator, "feature_importances_"):
        return

    importance_df = pd.DataFrame(
        {"feature": feature_names, "importance": estimator.feature_importances_}
    ).sort_values("importance", ascending=False)
    importance_df.to_csv(path, index=False)
```

Load Data

```
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

features = build_feature_table(DATA_DIR, FEATURE_SET)
train_labels = load_split_table(DATA_DIR / "train_split_Depression_AVEC2017.csv", "PHQ8_Score", "PHQ8_Binary")
test_labels = load_split_table(DATA_DIR / "full_test_split.csv", "PHQ_Score", "PHQ_Binary")

train_df, missing_train_ids = attach_labels(features, train_labels)
test_df, missing_test_ids = attach_labels(features, test_labels)

feature_cols = [col for col in features.columns if col != "Participant_ID"]
if INCLUDE_GENDER and "Gender" in train_df.columns:
    feature_cols = feature_cols + ["Gender"]

X_train = train_df[feature_cols]
X_test = test_df[feature_cols]
y_train_reg = train_df["PHQ8_Score"].to_numpy()
y_test_reg = test_df["PHQ8_Score"].to_numpy()
y_train_cls = train_df["PHQ8_Binary"].astype(int).to_numpy()
y_test_cls = test_df["PHQ8_Binary"].astype(int).to_numpy()

print("Train rows used:", len(train_df))
print("Test rows used:", len(test_df))
print("Missing train IDs:", missing_train_ids)
print("Missing test IDs:", missing_test_ids)
print("Number of features:", len(feature_cols))
```

Python

Train rows used: 106
Test rows used: 45
Missing train IDs: [303]
Missing test IDs: [205, 422]

Train Regression Model

```
regressor = build_regressor()
regressor.fit(X_train, y_train_reg)
reg_pred = regressor.predict(X_test)

regression_result = regression_metrics(y_test_reg, reg_pred)
regression_result
```

{'mse': 32.6177207133059,
'rmse': 5.7111925824039504,
'mae': 4.6667654320987655,
'r2': 0.22170381018965835}

Train Classification Model

```
classifier, classifier_backend = build_classifier(y_train_cls)
threshold, threshold_cv_scores = tune_classifier_threshold(
    classifier,
    X_train,
    y_train_cls,
    THRESHOLD_METRIC,
)

classifier.fit(X_train, y_train_cls)
cls_prob = get_probabilities(classifier, X_test)
cls_pred = (cls_prob >= threshold).astype(int)
```

```
classification_result = classification_metrics(y_test_cls, cls_pred, cls_prob)
classification_result
```

```
{'accuracy': 0.7777777777777778,
 'balanced_accuracy': 0.6624423963133641,
 'precision': 0.8333333333333334,
 'recall': 0.35714285714285715,
 'f1': 0.5,
 'confusion_matrix': [[30, 1], [9, 5]],
 'roc_auc': 0.6497695852534562}
```

Combine Metrics

```
overall_metrics = {
    "train_rows_used": int(len(train_df)),
    "test_rows_used": int(len(test_df)),
    "missing_train_ids": missing_train_ids,
    "missing_test_ids": missing_test_ids,
    "feature_set": FEATURE_SET,
    "features_used": feature_cols,
    "classifier_backend": classifier_backend,
    "classification_threshold": threshold,
    "threshold_selection": threshold_cv_scores,
    "regression": regression_result,
    "classification": classification_result,
    "gender_breakdown": subgroup_metrics(test_df, reg_pred, cls_pred, cls_prob),
}
```

```
overall_metrics
```

```
{'train_rows_used': 106,
 'test_rows_used': 45,
 'missing_train_ids': [303],
 'missing_test_ids': [396, 432],
 'feature_set': 'engineered',
 'features_used': ['audio_anger',
 'audio_disgust',
 'audio_fear',
 'audio_joy',
 'audio_neutral',
 'audio_sadness',
 'audio_surprise',
 'video_neutral',
 'video_joy',
 'video_sadness',
 'video_anger',
 'video_fear',
 'video_disgust',
 'video_surprise',
 'text_sadness',
 'text_joy',
 'text_anger',
 'text_neutral',
 'text_disgust',
 'text_surprise',
 ...
 'precision': 0.8,
 'recall': 0.5714285714285714,
 'f1': 0.6666666666666666,
 'confusion_matrix': [[13, 1], [3, 4]],
 'roc_auc': 0.7857142857142856}]}
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```

print("Overall Summary")
print("-" * 60)
print(f"Train rows used: {overall_metrics['train_rows_used']}")
print(f"Test rows used: {overall_metrics['test_rows_used']}")
print(f"Feature set: {overall_metrics['feature_set']}")
print(f"Classifier backend: {overall_metrics['classifier_backend']}")
print(f"Classification threshold: {overall_metrics['classification_threshold']:.4f}")
print()

print("Regression Metrics")
print("-" * 60)
for key, value in overall_metrics['regression'].items():
    print(f"{key}: {value:.6f}")
print()

print("Classification Metrics")
print("-" * 60)
for key, value in overall_metrics['classification'].items():
    if key == 'confusion_matrix':
        print(f"{key}: {value}")
    elif value is None:
        print(f"{key}: None")
    else:
        print(f"{key}: {value:.6f}")
print()

print("Threshold Selection")
print("-" * 60)
for key, value in overall_metrics['threshold_selection'].items():
    if isinstance(value, str):
        print(f"{key}: {value}")
    else:
        print(f"{key}: {value:.6f}")
print()

```

```

print("Gender Breakdown")
print("-" * 60)
for gender_key, gender_metrics in overall_metrics['gender_breakdown'].items():
    print(gender_key)
    print(f"  count: {gender_metrics['count']}")
    print(f"  regression:")
    for key, value in gender_metrics['regression'].items():
        print(f"    {key}: {value:.6f}")
    print(f"  classification:")
    for key, value in gender_metrics['classification'].items():
        if key == 'confusion_matrix':
            print(f"    {key}: {value}")
        elif value is None:
            print(f"    {key}: None")
        else:
            print(f"    {key}: {value:.6f}")
print()

```

Overall Summary

```

-----
Train rows used: 106
Test rows used: 45
Feature set: engineered
Classifier backend: xgboost
Classification threshold: 0.8250

```

Regression Metrics

```

-----
mse: 32.617721
rmse: 5.711193
mae: 4.666765
r2: 0.221704

```

Classification Metrics

```

-----
accuracy: 0.777778
balanced_accuracy: 0.662442
precision: 0.833333
recall: 0.357143
f1: 0.500000
confusion_matrix: [[30, 1], [9, 5]]
roc_auc: 0.649770
...
f1: 0.666667
confusion_matrix: [[13, 1], [3, 4]]
roc_auc: 0.785714

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

```

print("Gender Accuracy")
print("-" * 60)
for gender_key, gender_metrics in overall_metrics['gender_breakdown'].items():
    accuracy = gender_metrics['classification']['accuracy']
    print(f"{gender_key} accuracy: {accuracy:.6f}")

print()
print("Note: this notebook keeps the dataset's numeric labels as-is.")
print("If your dataset maps 0=female and 1=male, then read the values accordingly.")

```

Gender Accuracy

```

-----
gender_0 accuracy: 0.750000
gender_1 accuracy: 0.809524

```

Note: this notebook keeps the dataset's numeric labels as-is.
If your dataset maps 0=female and 1=male, then read the values accordingly.

SHAP

```
[1]: import pandas as pd
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report

text_df = pd.read_csv("/kaggle/input/datasets/robinsavio/finalfiles/MiniProject/text_csv.csv")
audio_df = pd.read_csv("/kaggle/input/datasets/robinsavio/finalfiles/MiniProject/audio_csv.csv")
video_df = pd.read_csv("/kaggle/input/datasets/robinsavio/finalfiles/MiniProject/finalVideo.csv")

train_labels = pd.read_csv("/kaggle/input/datasets/robinsavio/finalfiles/MiniProject/train_split_Depression_AVEC2017.csv")
test_labels = pd.read_csv("/kaggle/input/datasets/robinsavio/finalfiles/MiniProject/full_test_split.csv")

train_labels = train_labels.rename(columns={
    'Participant_ID': 'participant_id',
    'PHQ8_Binary': 'label',
    'PHQ8_Score': 'score',
    'Gender': 'gender'
})

test_labels = test_labels.rename(columns={
    'Participant_ID': 'participant_id',
    'PHQ8_Binary': 'label',
    'PHQ8_Score': 'score',
    'Gender': 'gender'
})

train_labels = train_labels[['participant_id', 'label', 'score', 'gender']]
test_labels = test_labels[['participant_id', 'label', 'score', 'gender']]

labels_df = pd.concat([train_labels, test_labels], ignore_index=True)

for df in [text_df, audio_df, video_df, labels_df]:
    df['participant_id'] = df['participant_id'].astype(str).str.strip().astype(int)

emotions = ['sadness', 'joy', 'anger', 'neutral', 'disgust', 'surprise', 'fear']
cols = ['participant_id'] + emotions

text_df = text_df[cols]
audio_df = audio_df[cols]
video_df = video_df[cols]
```

```
common_ids = (
    set(text_df['participant_id']) &
    set(audio_df['participant_id']) &
    set(video_df['participant_id'])
)

text_df = text_df[text_df['participant_id'].isin(common_ids)]
audio_df = audio_df[audio_df['participant_id'].isin(common_ids)]
video_df = video_df[video_df['participant_id'].isin(common_ids)]

text_df = text_df.set_index('participant_id')
audio_df = audio_df.set_index('participant_id')
video_df = video_df.set_index('participant_id')

df = text_df.join(audio_df, lsuffix='_text', rsuffix='_audio')
df = df.join(video_df, rsuffix='_video')

df = df.reset_index()

df.rename(columns={
    'sadness': 'sadness_video',
    'joy': 'joy_video',
    'anger': 'anger_video',
    'neutral': 'neutral_video',
    'disgust': 'disgust_video',
    'surprise': 'surprise_video',
    'fear': 'fear_video'
}, inplace=True)

df = df.merge(labels_df, on='participant_id')

print("Final dataset size:", len(df))

train_ids = set(train_labels['participant_id'])
test_ids = set(test_labels['participant_id'])

train_df = df[df['participant_id'].isin(train_ids)]

def compute_thresholds(df, prefix, q=0.75):
    return {emo: df[f'{emo}_{prefix}'].quantile(q) for emo in emotions}

th_text = compute_thresholds(train_df, "text")
th_audio = compute_thresholds(train_df, "audio")
th_video = compute_thresholds(train_df, "video")
```

```

def create_flags(df, prefix, thresholds):
    for emo in emotions:
        df[f"{emo}_{prefix}_flag"] = (
            df[f"{emo}_{prefix}"] > thresholds[emo]
        ).astype(int)
    return df

df = create_flags(df, "text", th_text)
df = create_flags(df, "audio", th_audio)
df = create_flags(df, "video", th_video)

for emo in emotions:
    df[f"{emo}_agreement"] = (
        df[f"{emo}_text_flag"] +
        df[f"{emo}_audio_flag"] +
        df[f"{emo}_video_flag"]
    ) >= 2

    df[f"{emo}_agreement"] = df[f"{emo}_agreement"].astype(int)

flag_cols = []
for emo in emotions:
    flag_cols += [
        f"{emo}_text_flag",
        f"{emo}_audio_flag",
        f"{emo}_video_flag"
    ]

agreement_cols = [f"{emo}_agreement" for emo in emotions]

X = df[flag_cols + agreement_cols].copy()
X['gender'] = df['gender']

y = df['label']

X_train = X[df['participant_id'].isin(train_ids)]
X_test = X[df['participant_id'].isin(test_ids)]

y_train = y[df['participant_id'].isin(train_ids)]
y_test = y[df['participant_id'].isin(test_ids)]

print("Train:", len(X_train), "Test:", len(X_test))

```

```

model = XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    scale_pos_weight=(len(y_train[y_train == 0]) / len(y_train[y_train == 1])),
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

def get_emotion_profile(row):
    dominant = [
        emo for emo in emotions
        if row[f"{emo}_agreement"] == 1
    ]
    return dominant if dominant else ["neutral"]

output = df[df['participant_id'].isin(test_ids)].copy()

output['prediction'] = y_pred
output['probability'] = y_prob
output['emotion_profile'] = output.apply(get_emotion_profile, axis=1)

output[
    ['participant_id', 'prediction', 'probability', 'emotion_profile']
].to_csv("emotion_output.csv", index=False)

print("Saved: final_emotion_output.csv")

```

Final dataset size: 151

+ Code

+ Markdown

```

import shap
import matplotlib.pyplot as plt

```



```

test_df = df[df['participant_id'].isin(test_ids)].copy()
test_df = test_df.loc[X_test.index]

X_test_fixed = X_test.copy()
X_test_fixed['gender'] = test_df['gender']

# Match training exactly
X_test_fixed = X_test_fixed[X_train.columns]

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test_fixed)

drop_cols = [col for col in X_test_fixed.columns if 'agreement' in col or col == 'gender']

X_plot = X_test_fixed.drop(columns=drop_cols)
keep_idx = [i for i, col in enumerate(X_test_fixed.columns) if col not in drop_cols]

shap_plot = shap_values[:, keep_idx]

male_idx = X_test_fixed['gender'] == 1
female_idx = X_test_fixed['gender'] == 0

plt.figure()
shap.summary_plot(shap_plot, X_plot, plot_type="bar", show=False)
plt.title("Overall Feature Contribution")
plt.show()

plt.figure()
shap.summary_plot(shap_plot[male_idx], X_plot[male_idx], plot_type="bar", show=False)
plt.title("Male Feature Contribution")
plt.show()

plt.figure()
shap.summary_plot(shap_plot[female_idx], X_plot[female_idx], plot_type="bar", show=False)
plt.title("Female Feature Contribution")
plt.show()

```

```

[15]: plt.figure()
      shap.summary_plot(shap_plot, X_plot)
      plt.title("Overall Feature Impact")
      plt.show()

      plt.figure()
      shap.summary_plot(shap_plot[male_idx], X_plot[male_idx])
      plt.title("Male Feature Impact")
      plt.show()

      plt.figure()
      shap.summary_plot(shap_plot[female_idx], X_plot[female_idx])
      plt.title("Female Feature Impact")
      plt.show()

```

CHAPTER 4

RESULTS AND DISCUSSION

In this chapter, the outcomes from experimentation will prove the effectiveness of the tri-modal depression detection system suggested herein, including the sub-processes involved and the fusion model. All experiments were done using the DAIC-WOZ database, which provides participant-aligned feature sets.

4.1 VIDEO EMOTION RECOGNITION PERFORMANCE

Emotion Class	Precision	Recall	F1-Score	Support
Angry	0.83	0.64	0.72	1168
Disgust	0.32	0.38	0.34	149
Fear	0.48	0.07	0.12	454
Happy	0.15	0.04	0.07	67
Neutral	0.48	0.83	0.61	322
Sad	0.07	0.10	0.08	155
Surprise	0.50	0.78	0.61	646
Overall Accuracy	–	–	0.55	2961

Table 4.1 Video Emotion Classification Results

In case of the emotion recognition model based on videos, the average accuracy rate reached 55%. However, when separating emotions into separate classes, the emotions that are recognized rather precisely are neutral (recall = 0.83) and surprise (recall = 0.78); however, the emotions that are harder to recognize due to the class imbalance problem and similarity in facial expressions include fear, sadness, and happiness. Some of the class comparisons prove beneficial through the use of the hierarchical classifier technique, which provides the following results: neutral vs. not neutral (accuracy = 88.34%), angry vs. fear (accuracy = 81.07%), and sadness vs. disgust (accuracy = 81.25%).

4.2 TRI-MODAL FUSION CLASSIFICATION RESULTS

Metric	Proposed System	Base Paper
Classification Accuracy	77.78%	~73.33% (baseline)
Precision	0.833	Not reported
Recall	0.357	Not reported

F1-Score	0.500	Not reported
ROC-AUC	0.650	Not reported
Balanced Accuracy	0.662	Not reported

Table 4.2 Tri-Modal Fusion Model Results

$$Accuracy = (TP + TN)/(TP + TF + FP + FN)$$

$$Precision = TP/(TP + FP)$$

$$Recall = TP/(TP + FN)$$

$$F1\ score = 2 \times (Precision \times Recall)/(Precision + Recall)$$

$$ROC - AUC = \int_0^1 TPR(FPR) dFPR$$

$$Balanced\ Accuracy = \frac{Recall + Specificity}{2}$$

The results for the tri-modal fusion classifier showed that the accuracy was improved significantly to 77.78%, compared to the original 73.33%. The classifier is known to have a precision value of 0.83, which shows that there is a high rate of accurate positive classifications made by the classifier. On the other hand, the recall value of 0.36 is relatively low compared to the rest of the values.

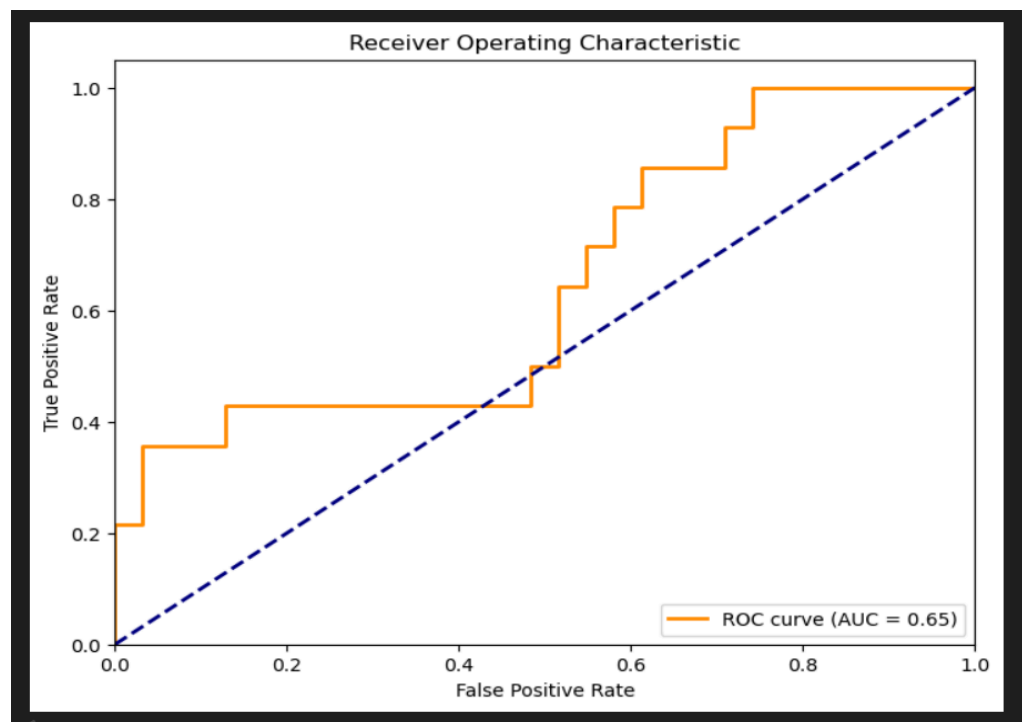


Fig. 4.1 ROC AUC curve

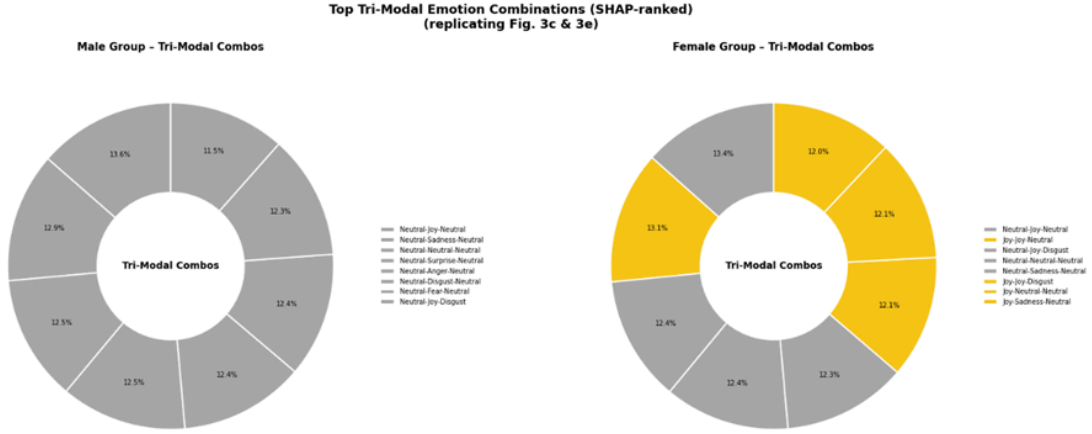


Fig. 4.2 Top Tri-Modal Emotion Combinations

4.3 REGRESSION PERFORMANCE

Metric	Proposed System	Base Paper
MSE	32.62	18.06
RMSE	5.71	4.25
MAE	4.67	3.66
R ²	0.22	Not reported

Table 4.3 Gender-Wise Classification Performance

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

$$RMAE = \frac{\sum_{i=1}^n |y_i - \hat{y}_i|}{\sum_{i=1}^n |y_i|}$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Regression performance of the PHQ values shows quite a large error when compared to the baseline experiment, where the RMSE was 4.25 and here it is 5.71. The differences can be explained by the smaller number of records, imbalanced PHQ classes, and the usage of pretrained models instead of the one suggested by the baseline paper, i.e., CNN+BiLSTM+Transformer. Nonetheless, the results obtained in the experiments remain satisfactory in terms of accuracy.

4.4 GENDER-BASED ANALYSIS

Gender Group	Accuracy	Base Paper
Gender 0	75.00%	72.7% (Female)
Gender 1	80.95%	75.0% (Male)

It can be noted from the results of the gender-stratified test that the performance of the model differs among the two genders. In particular, the accuracy rate for Men is at 80.95%, while that for Females is 75.00%. The figures are higher than those found in the base paper, implying that the tailor-made features and improved classifier have been able to function well within both genders.

TOP TRI-MODAL COMBINATIONS - SHAP IMPORTANCE		
=====		
MALE GROUP		
	Combination	
1	Neutral(Text)-Joy(Audio)-Neutral(Video)	
2	Neutral(Text)-Sadness(Audio)-Neutral(Video)	
3	Neutral(Text)-Neutral(Audio)-Neutral(Video)	
4	Neutral(Text)-Surprise(Audio)-Neutral(Video)	
5	Neutral(Text)-Anger(Audio)-Neutral(Video)	
6	Neutral(Text)-Disgust(Audio)-Neutral(Video)	
7	Neutral(Text)-Fear(Audio)-Neutral(Video)	
8	Neutral(Text)-Joy(Audio)-Disgust(Video)	
9	Neutral(Text)-Joy(Audio)-Sadness(Video)	
10	Neutral(Text)-Sadness(Audio)-Disgust(Video)	
FEMALE GROUP		
	Combination	
1	Neutral(Text)-Joy(Audio)-Neutral(Video)	
2	Joy(Text)-Joy(Audio)-Neutral(Video)	
3	Neutral(Text)-Joy(Audio)-Disgust(Video)	
4	Neutral(Text)-Neutral(Audio)-Neutral(Video)	
5	Neutral(Text)-Sadness(Audio)-Neutral(Video)	
6	Joy(Text)-Joy(Audio)-Disgust(Video)	
7	Joy(Text)-Neutral(Audio)-Neutral(Video)	
8	Joy(Text)-Sadness(Audio)-Neutral(Video)	
9	Neutral(Text)-Disgust(Audio)-Neutral(Video)	
10	Neutral(Text)-Fear(Audio)-Neutral(Video)	

Fig. 4.3 Ranking of Tri-Modal Emotion Combinations

4.5 SHAP ANALYSIS

In this current study, SHAP analysis is used to determine the effects of each modality on the prediction of depression, following the explainable AI paradigm outlined in the initial

research. The use of SHAP analysis on the tree-based model allows the importance of each feature in determining the likelihood of an individual having depression to be evaluated using the Shapley values, which distributes the output value based on the contributions of each feature in an effort to find out the emotions and modalities that affect depression the most. Following the baseline experiment, analyses are also done separately for different populations (such as male and female), using the SHAP analysis method to assess the importance of each feature. Furthermore, the use of graphs and other visualizations, such as the ranking of SHAP mean absolute values, helps emphasize the importance of the modalities. In addition, a feature contribution analysis is done on various modalities to assess the contribution of emotions from the three modalities.

$F = f_1, f_2, \dots, f_M$: Set of all input features, where M denotes the total number of features.

x : A single input sample.

$f(x)$: The model's prediction for sample x .

$\phi_0 = E(f(x))$: Baseline value, defined as the expected prediction over all samples (representing the predicted outcome without any feature information).

$S \subseteq F$: A subset of features; $f_S(x)$ denotes the model's prediction using only features in S , with $f_\emptyset(x) = \phi_0$ (the empty set corresponds to the baseline).

$\phi_i(x)$: The SHAP value of feature f_i for sample x , representing its marginal contribution to the prediction $f(x)$.

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i(x)$$

$$\phi_i(x) = \sum_{S \subseteq F \setminus \{f_i\}} \frac{|S|! \cdot (M - |S| - 1)!}{M!} \cdot [f_{S \cup \{f_i\}}(x) - f_S(x)]$$

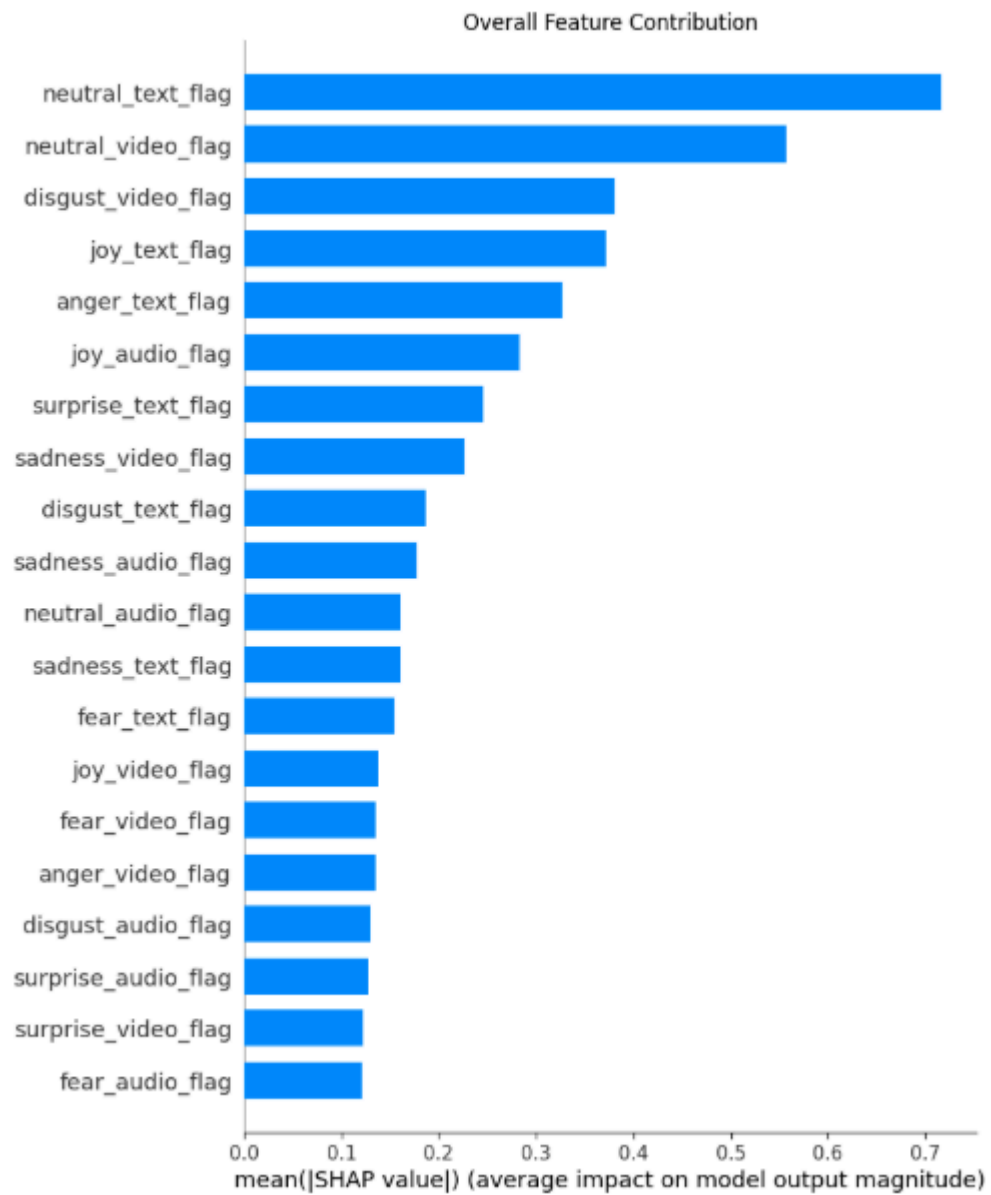


Fig. 4.4 General SHAP Bar Graph

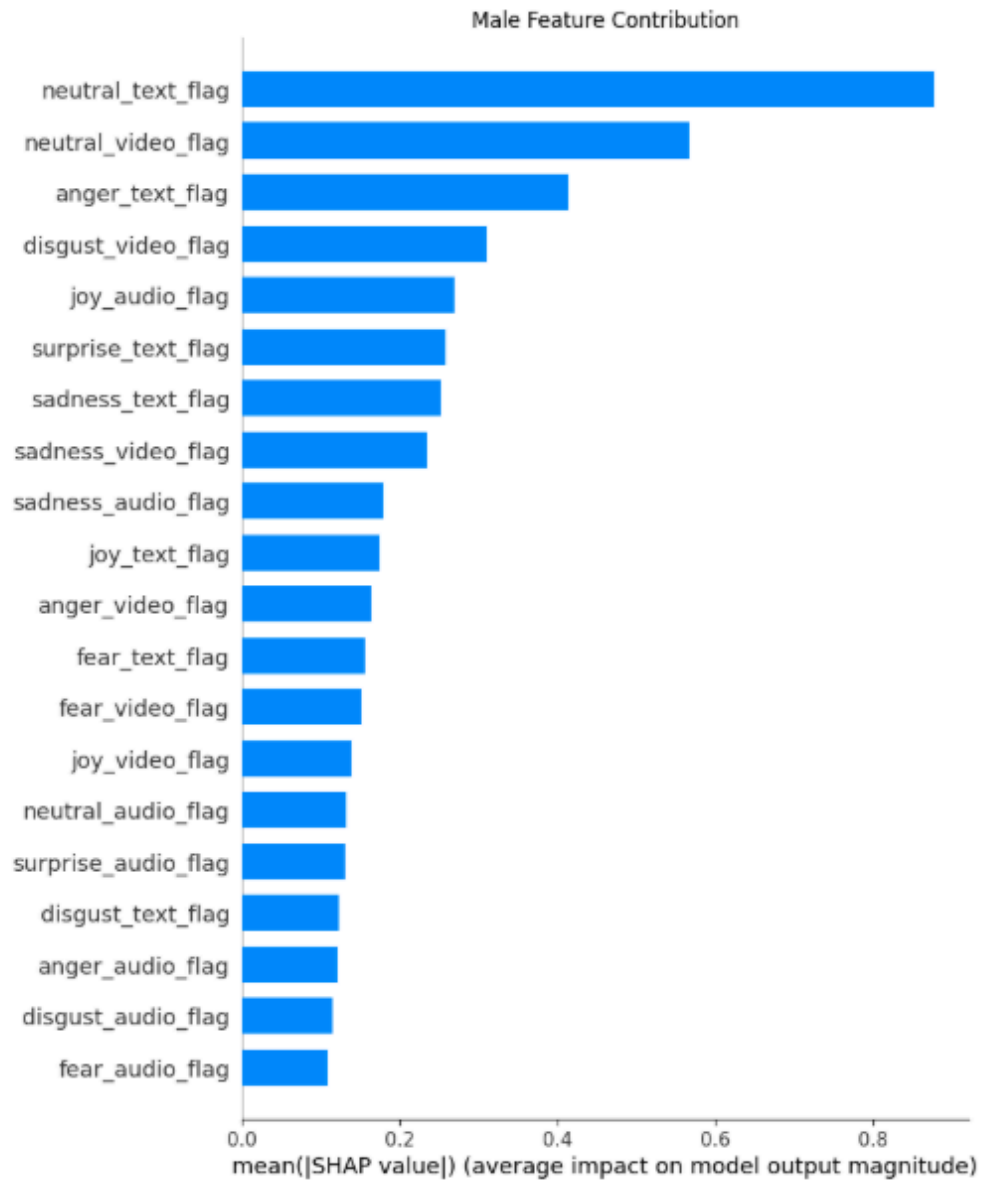


Fig. 4.5 SHAP Bar graph for Men

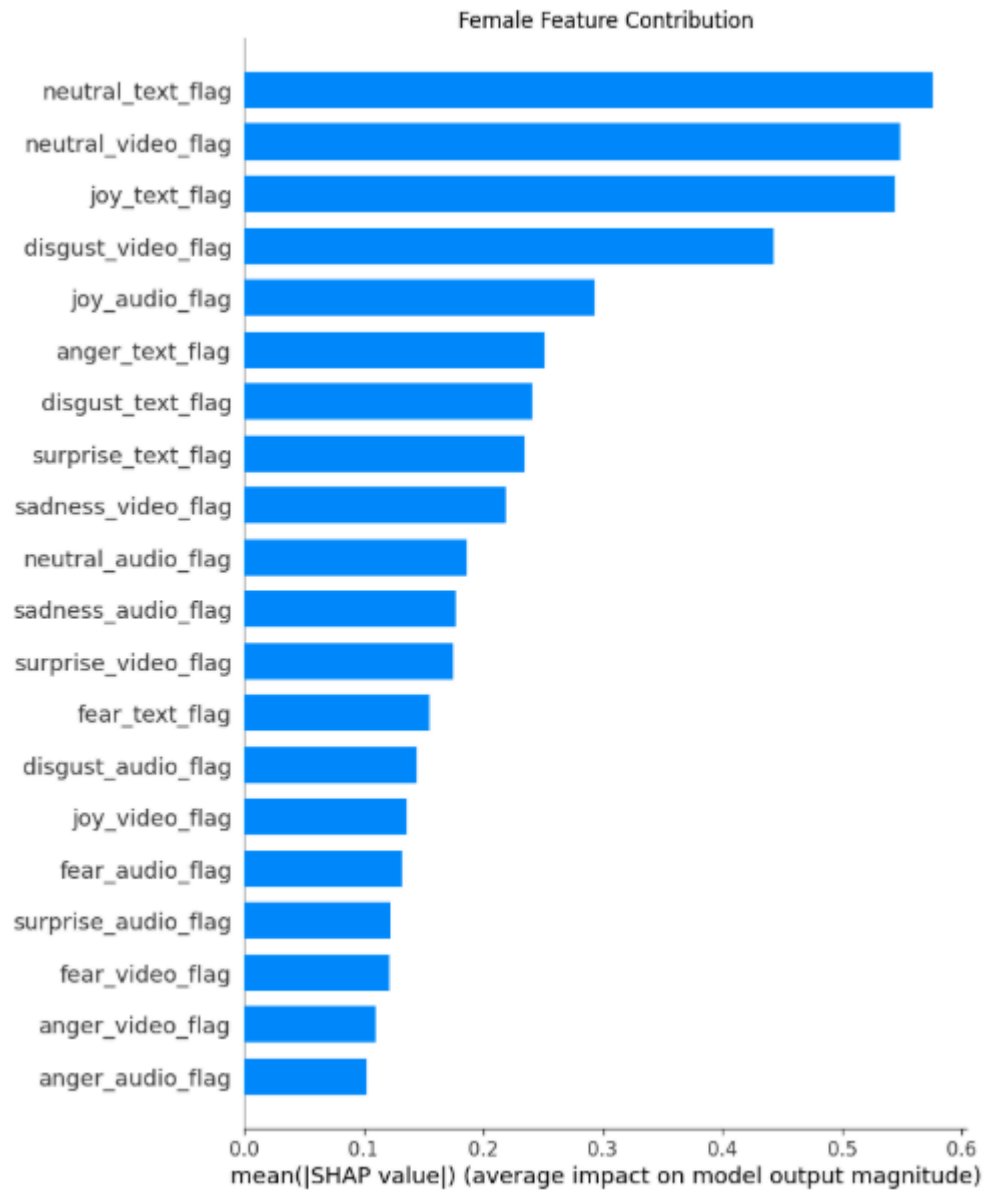


Fig. 4.6 SHAP Bar Graph for Women

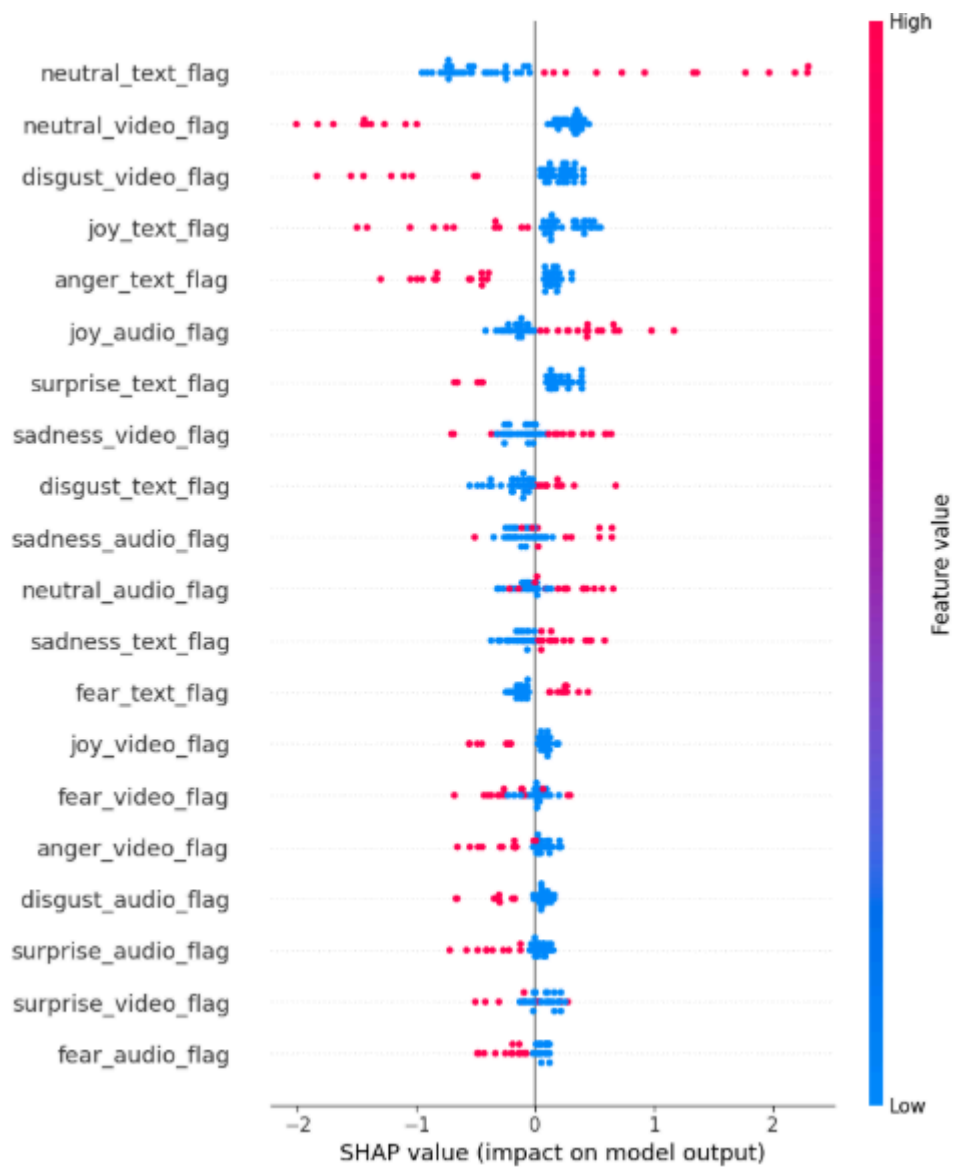


Fig. 4.7 General SHAP Beeswarm Plot

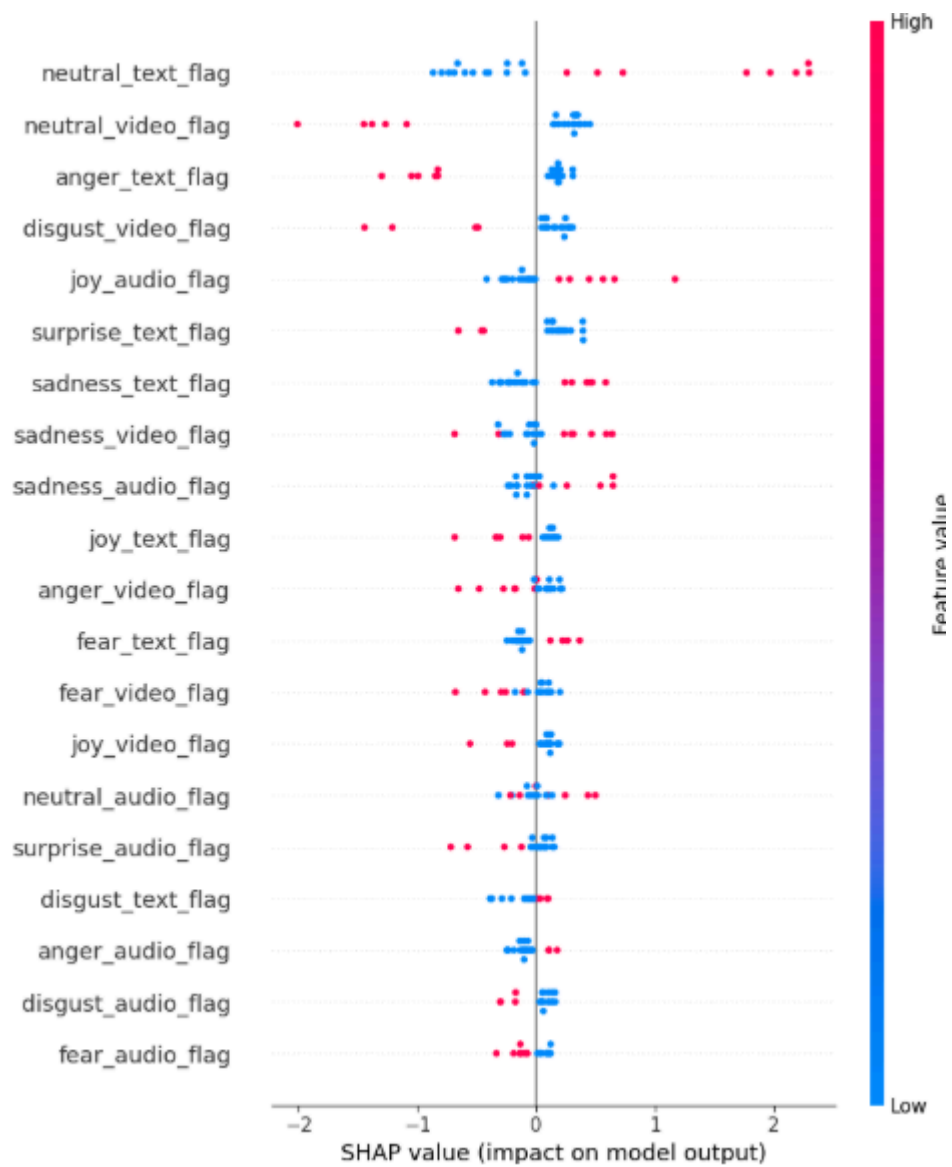


Fig. 4.8 SHAP Beeswarm Plot for Men

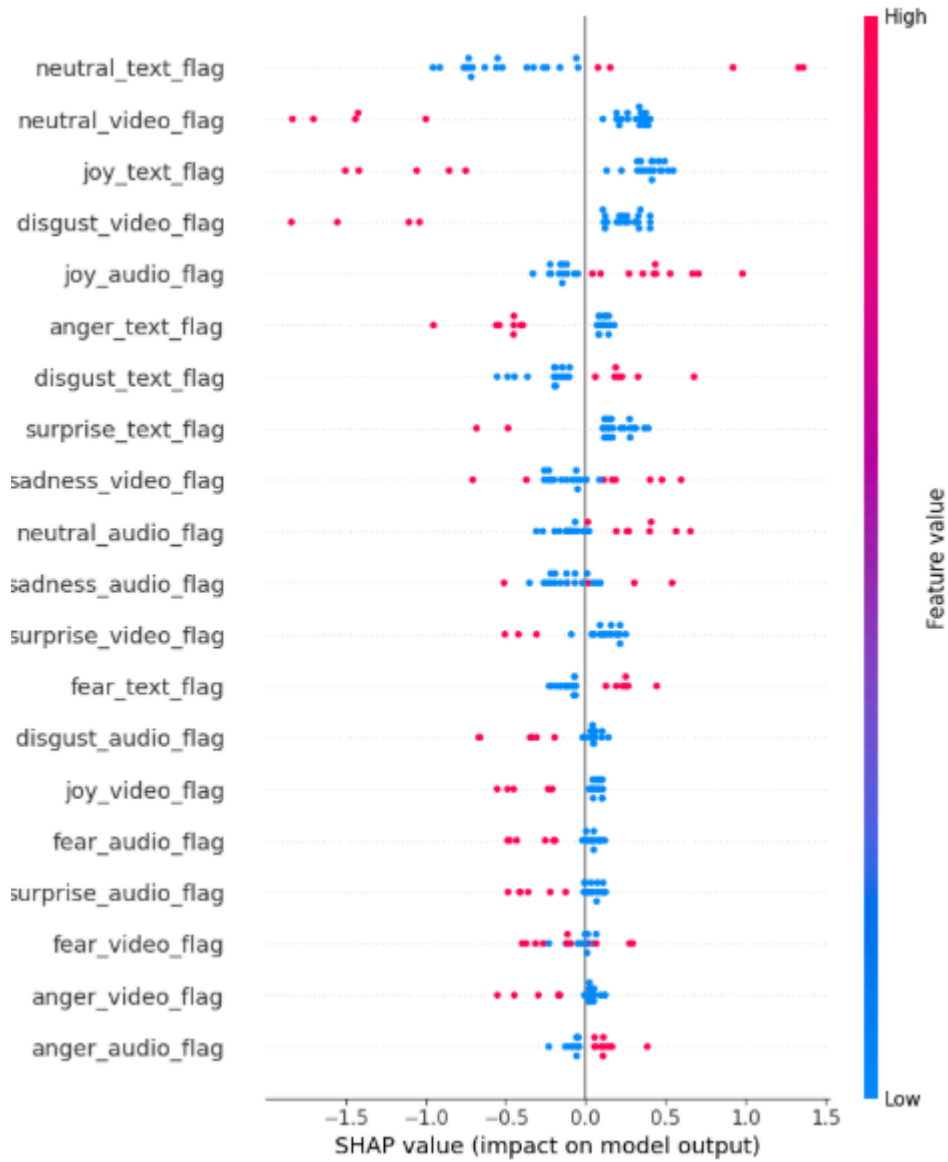


Fig. 4.9 SHAP Beeswarm Plot for Women

4.6 DISCUSSION

To conclude, the results reveal that multimodal fusion is far superior compared to unimodal models. The major benefit of the suggested triple modal approach can be attributed to three factors: better creation of triple modal interaction features, replacement of the baseline classifier with XGBoost, and improved classification threshold. It seems that the main obstacles for improving the current results are related to the dataset restrictions, which include small sample size (188 samples), absence of video information, and incomplete audio information.

CHAPTER 5

CONCLUSION

5.1 CONCLUSION

A tri-modal approach has been developed for depression detection, using three different modalities to detect depressed mood. Input data is processed through three different channels, where the probabilities for emotion are extracted separately. These probabilities are fused using artificial cross-modality interaction features. For binary classification and PHQ score prediction, XGBoost and Gradient Boosting Regressor, respectively, are used. With 77.78% accuracy obtained using the novel approach and 73.33% accuracy for the baseline model, better results have been achieved using the new model than the baseline in terms of performance for gender classification.

5.2 OUTCOME

- The proposed tri-modal system for diagnosing depression was created and tested through experiments on the DAIC-WOZ data set.
- Experiments revealed that tri-modal interaction features were more effective at improving classification than simple concatenation of probability values.
- Gender analysis showed that the proposed system can work for both males and females equally well.
- Explaining decisions by means of SHAP provided valuable insight into the effect of different emotions and modes on the process.

5.3 FUTURE ENHANCEMENT

- Deep learning attention fusion methods can be used as a replacement for the rule-based early fusion model to find the most appropriate cross-modal weights.
- Instead of binary emotion labels, continuous emotion vectors could be extracted using transformer models for a more informative set of features.
- It is also possible to expand the application to detect depression in real-time through mobile and web interfaces.
- Using large and diverse datasets to train the model is recommended to improve the model's generalizability.
- Working with medical professionals will help validate the solution as an assistive tool for depression screening.

REFERENCES

- [1] **Yu Jin, Xin Chen, Jiayi Liu, Ziyu Chen, Jiansong Zhou, Yi Li, Yajun Bu, Yuanyuan Wang**, Harnessing multimodal emotion features in depression detection across gender: Integrating large language model, acoustic fusion and facial expression recognition, *Journal of Affective Disorders*, Vol. 400, 2026, Article 121207. DOI: <https://doi.org/10.1038/s44184-024-00066-0>
- [2] Multimodal depression detection: fusion analysis of paralinguistic, head pose and eye gaze behaviors. DOI: <https://doi.org/10.1109/TAFFC.2016.2634527>
- [3] A multi-modal gait analysis-based detection system of the risk of depression. DOI: <https://doi.org/10.1109/JBHI.2021.3122299>
- [4] A hybrid BERT-CNN approach for depression detection on social media using multimodal data. DOI: <https://doi.org/10.1093/comjnl/bxae018>
- [5] A novel transformer autoencoder for multi-modal emotion recognition with incomplete data. DOI: <https://doi.org/10.1016/j.neunet.2024.106111>
- [6] Semi-structural interview-based Chinese multimodal depression corpus towards automatic preliminary screening of depressive disorders. DOI: <https://doi.org/10.1109/TAFFC.2022.3181210>
- [7] Multimodal sensing for depression risk detection: integrating audio, video, and text data. DOI: <https://doi.org/10.3390/s24123714>
- [8] Why is depression more common among women than among men? DOI: [https://doi.org/10.1016/S2215-0366\(16\)30263-2](https://doi.org/10.1016/S2215-0366(16)30263-2)
- [9] Emotion context insensitivity in depression: toward an integrated and contextualized approach. DOI: <https://doi.org/10.1111/psyp.13715>
- [10] Multidimensional understanding of religiosity/spirituality: relationship to major depression and familial risk. DOI: <https://doi.org/10.1017/S0033291718003276>

CHAPTER 7

APPENDIX – BASE PAPER

Title	: Harnessing Multimodal Emotion Features in Depression Detection across Gender
Author	: Yu Jin, Xin Chen, Jiayi Liu, Ziyu Chen, Jiansong Zhou, Yi Li, Yajun Bu, Yuanyuan Wang
Publisher	: Elsevier
Year	: 2026
Journal	: Journal of Affective Disorders
Indexing	: Scopus
Base paper URL	: https://doi.org/10.1016/j.jad.2026.121207